# OS_homework_4

这份文件是OS_homework_4 by计科210X 甘晴`void` 202108010XXX

文档设置了目录，可以通过目录快速跳转至答案部分。

## 第37章

运行程序wolf/OS-homework/file-disks/disk.py

解释一些参数

- -G可以查看可视化内容
- -c可以计算结果。
- -a 提供待访问的数组
- -S 将寻道速率改为不同值（第2题）
- -R 将旋转速率修改为不同值（第3题）
- -p 提供调度算法，默认FIFO，可以替换为SATF,SSTF等（第4题）
- -o 引入磁道偏移（第6题）

### ★这里千万要注意

出现`ModuleNotFoundError: No module named 'tkinter'`错误，表示python3没有安装可视化的tkinter包。

此时不能使用`pip install tkinter`来试图安装，即使换源也会显示失败。

一定要使用`sudo apt-get install python3-tk`来安装tkinter包。

具体原因未知，可能与tkinter的作用底层原理有关，但是这样操作可以完成运行的效果。

### 37.1

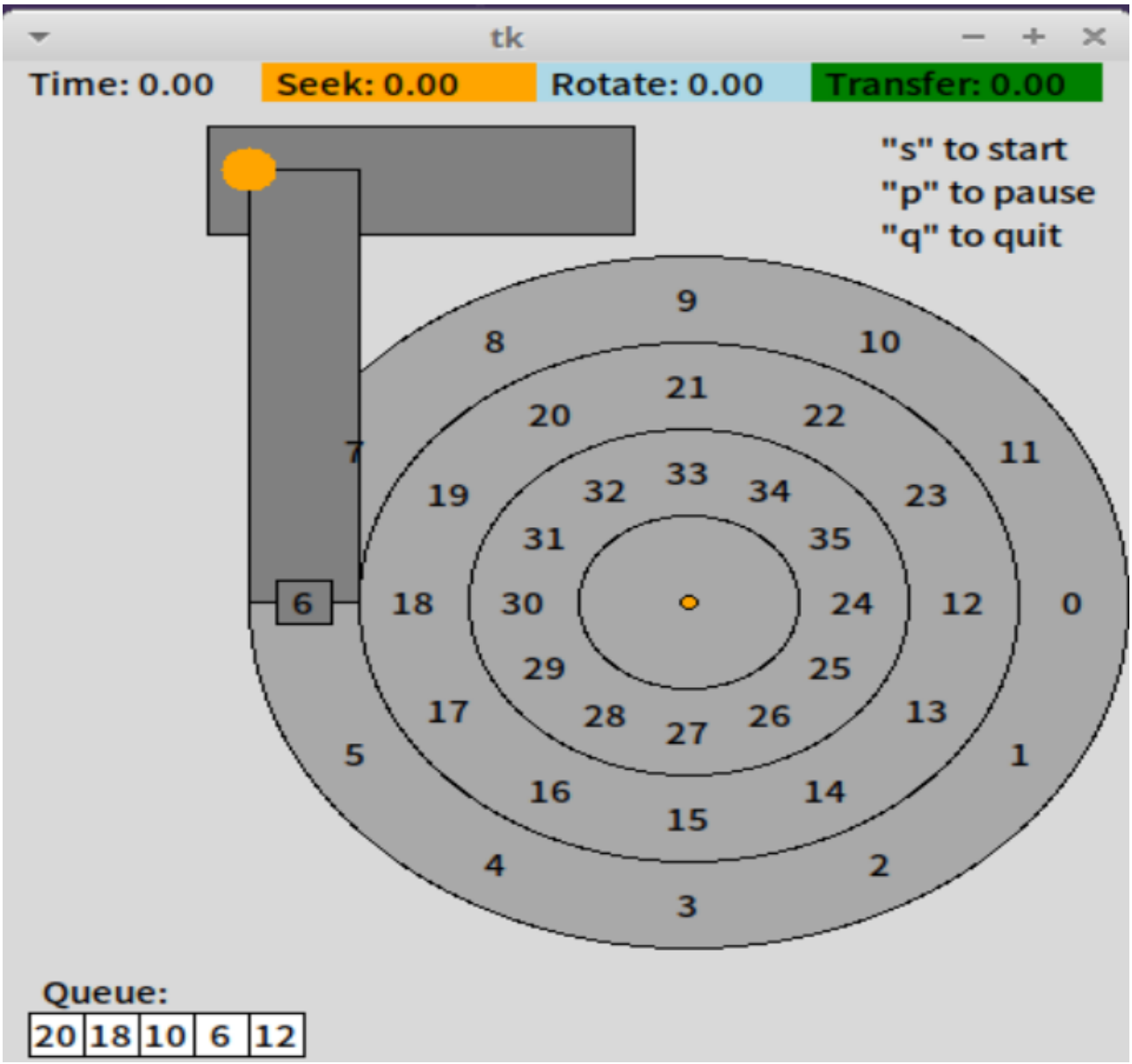
> 计算以下几组请求的寻道，旋转和传输时间 `-a 0; -a 6; -a 30; -a 7,30,8; -a 10,11,12,13`

★第一题真的很推荐自己理一遍，里面的细节真的很多，用心整理一遍后收获很大。

首先执行如下指令，查看初始状态下的磁盘情况。

```
python3 ./disk.py -G
```

运行截图如下：




可以看到，如图所示的情况。

下面分析我们关注的三个时间的计算方法。

T寻道

磁道之间的距离默认为40单位，而寻道速度为每单位时间移动一单位距离，因此相邻磁道的寻道时间为40单位时间。

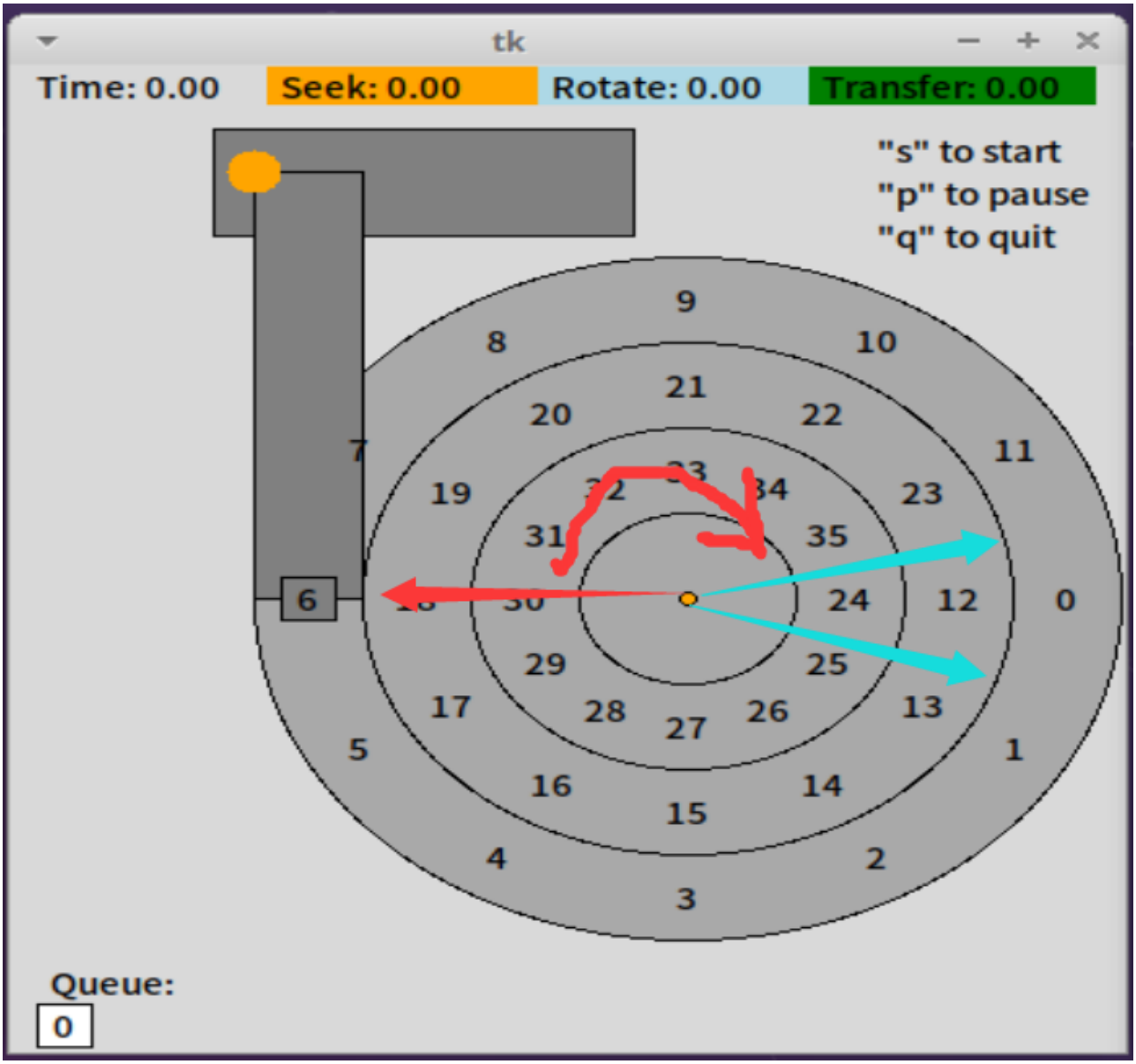
T旋转

旋转时间按照一度1个时间单位来计算。

T传输

假设旋转一度为1个时间单位，旋转完整一圈需要360个时间单位，而一个磁道上默认有12个扇区，每个扇区需要30个时间单元进行数据传输。

**-a 0**



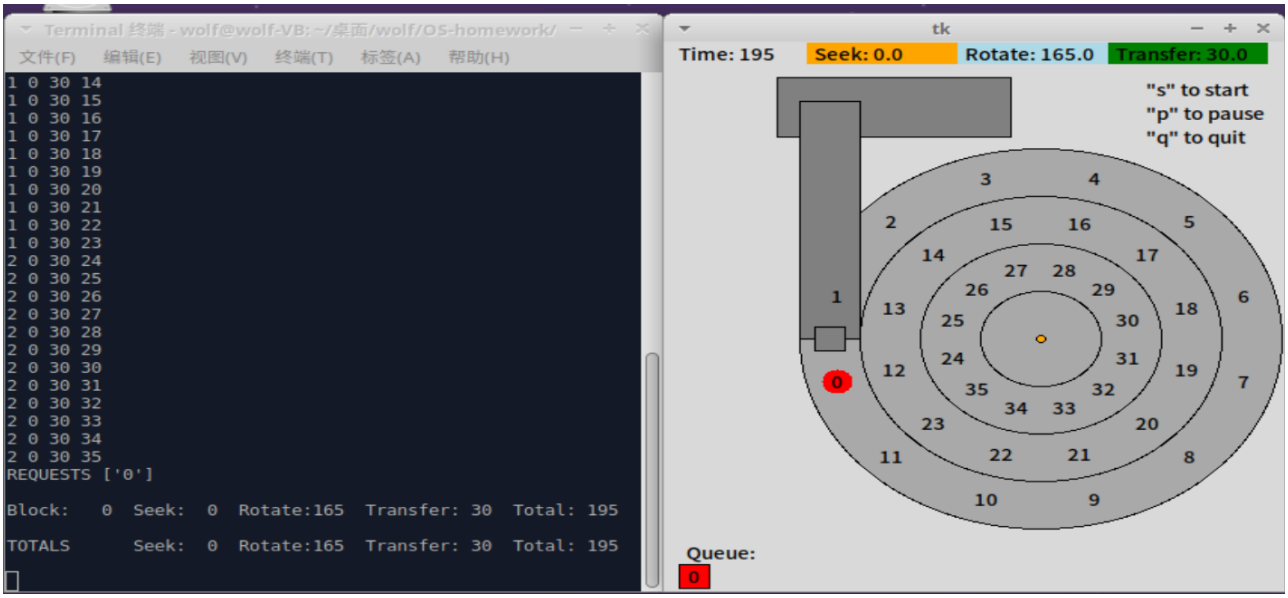

T寻道=0（在同一条磁道上）

T旋转=165（6到0之前：180-15=165）

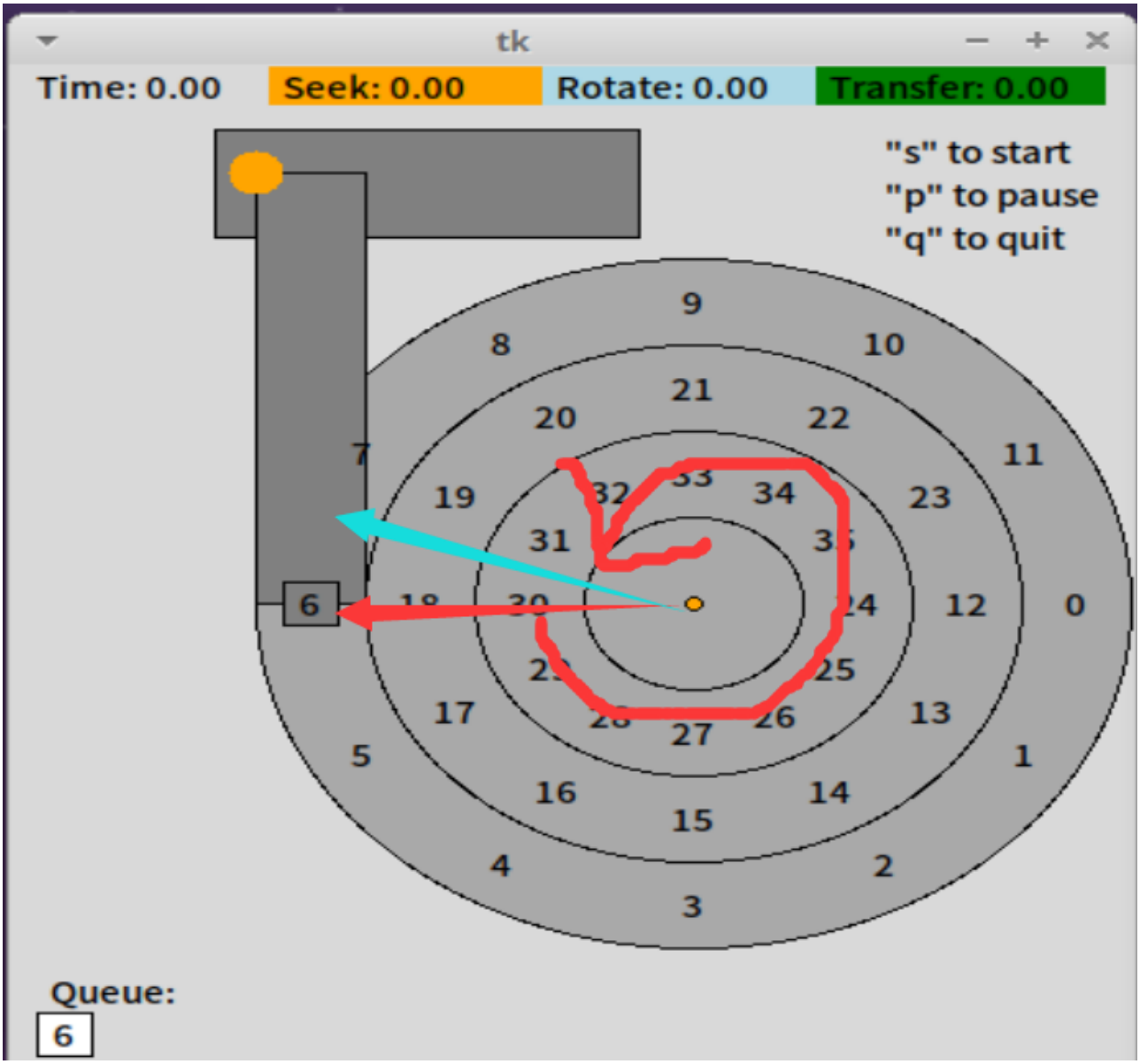
T传输=30

T总=T寻道+T旋转+T传输=195

模拟结果符合预期。




## -a 6



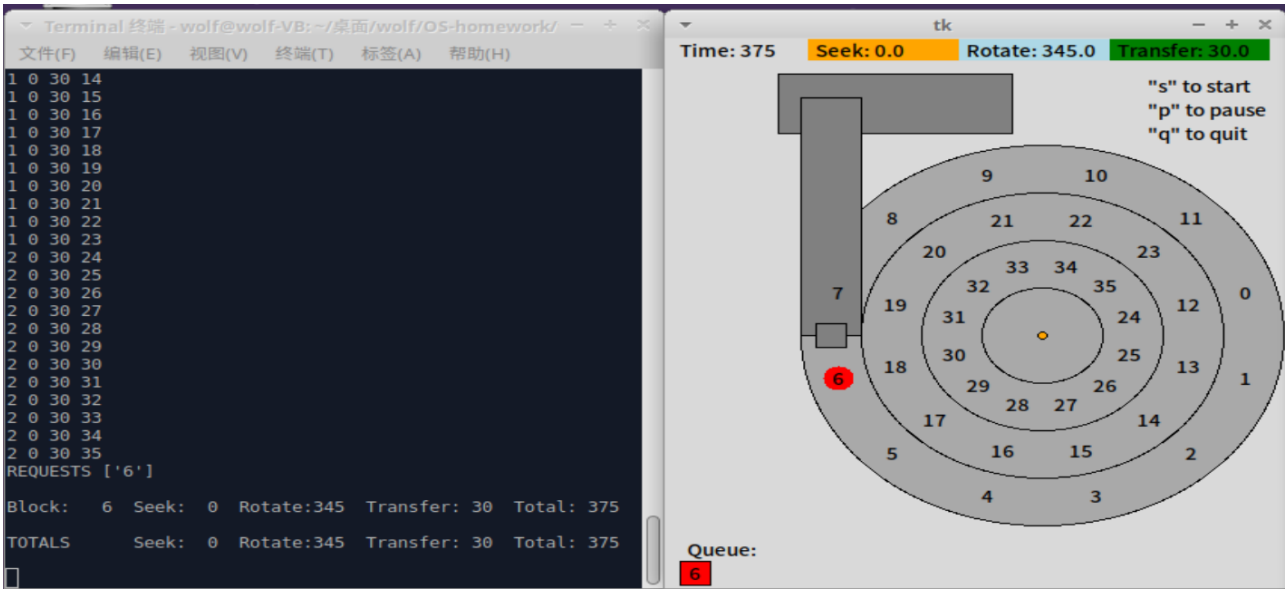

T寻道=0（在同一条磁道上）

T旋转=345（6到6之前：360-15=345）

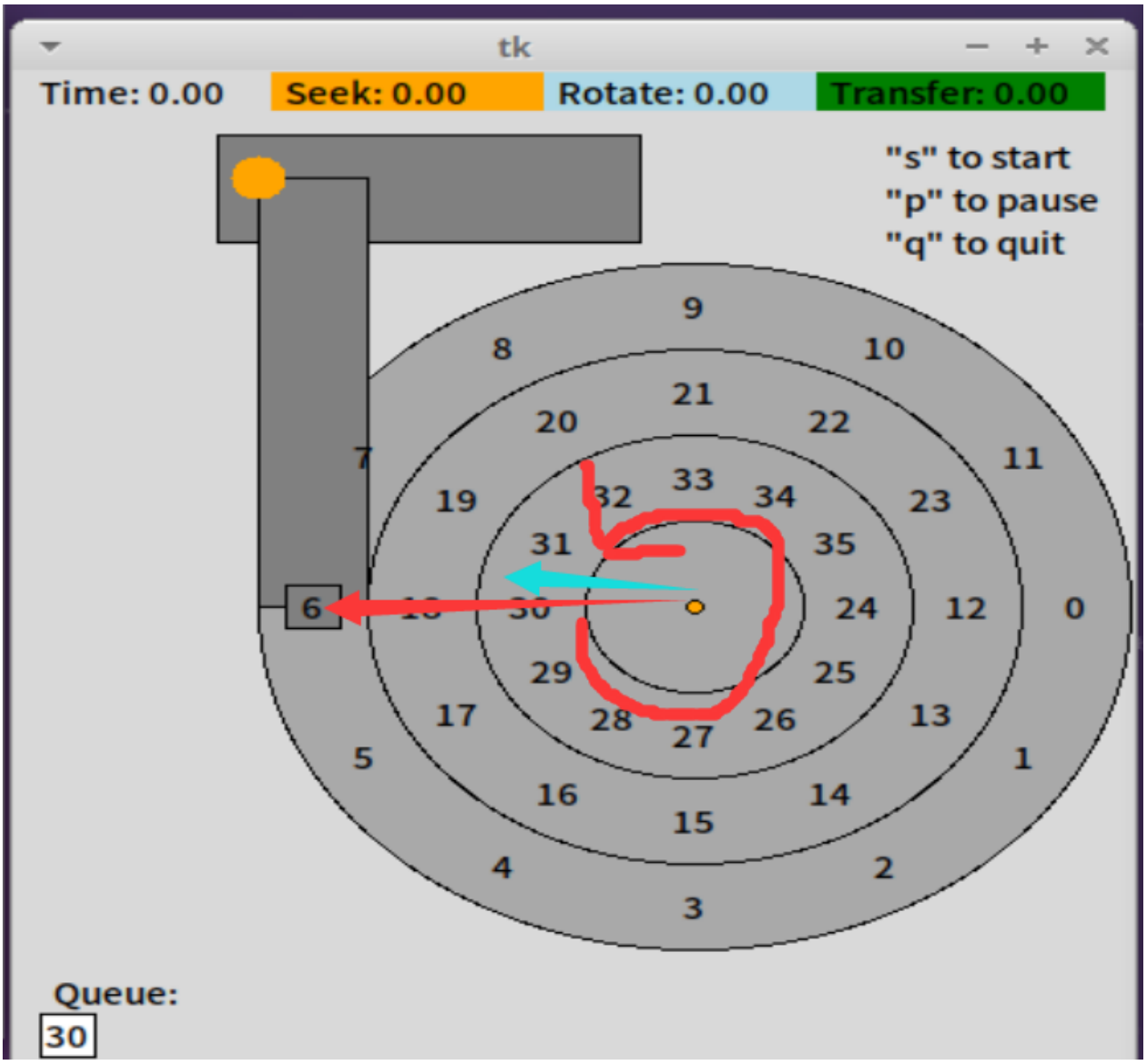
T传输=30

T总=T寻道+T旋转+T传输=375

模拟结果符合预期。




**-a 30**




T寻道=2*40=80（实际上在寻道的时候同时旋转也在进行）

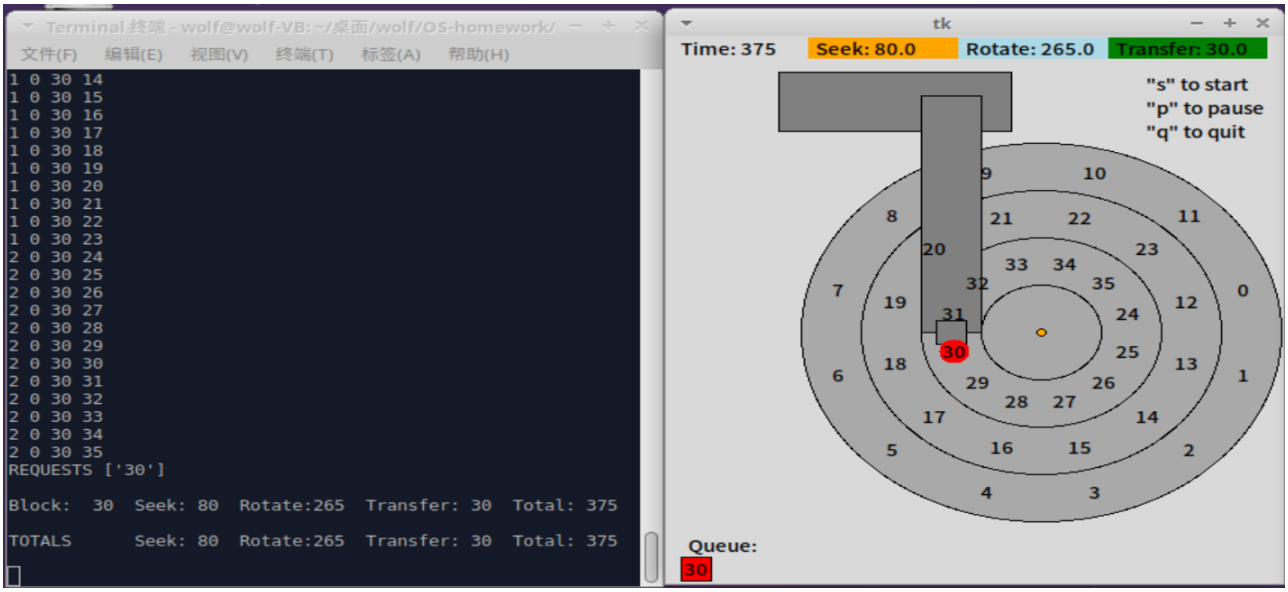
T旋转=345-80=265（角度相当于是6到6和7的中间：360-15=345，再减去寻道的时候不计入这一部分）

T传输=30

T总=T寻道+T旋转+T传输=375

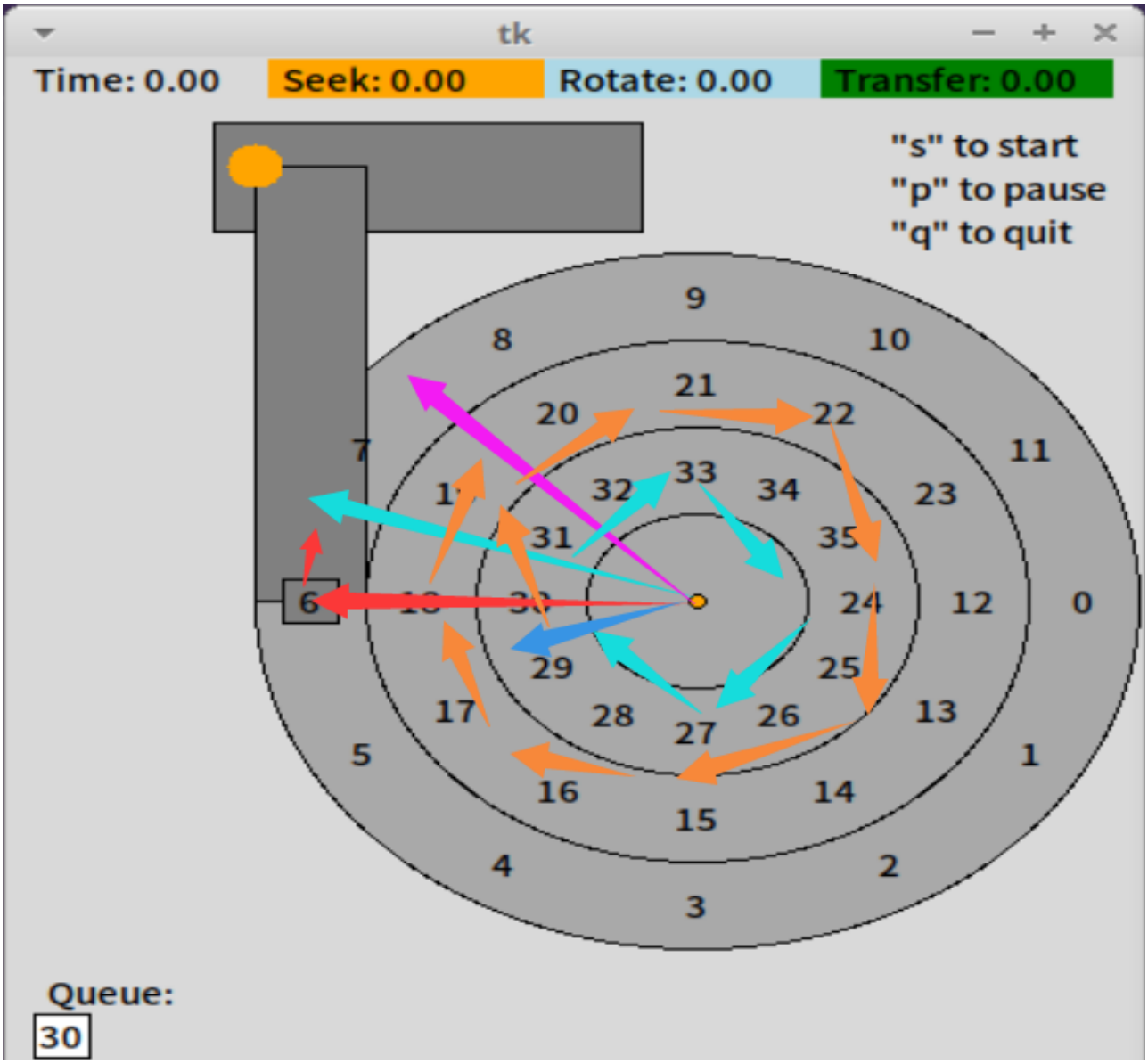
（与上一个实际上是一样的，这是因为寻道的同时也在旋转，实际上寻道时间被吞并了）

模拟结果符合预期。




## -a 7，30，8

这个模拟的应该是一个随机的读取情况。




这个不好直接分析（容易出错），需要分过程分析。

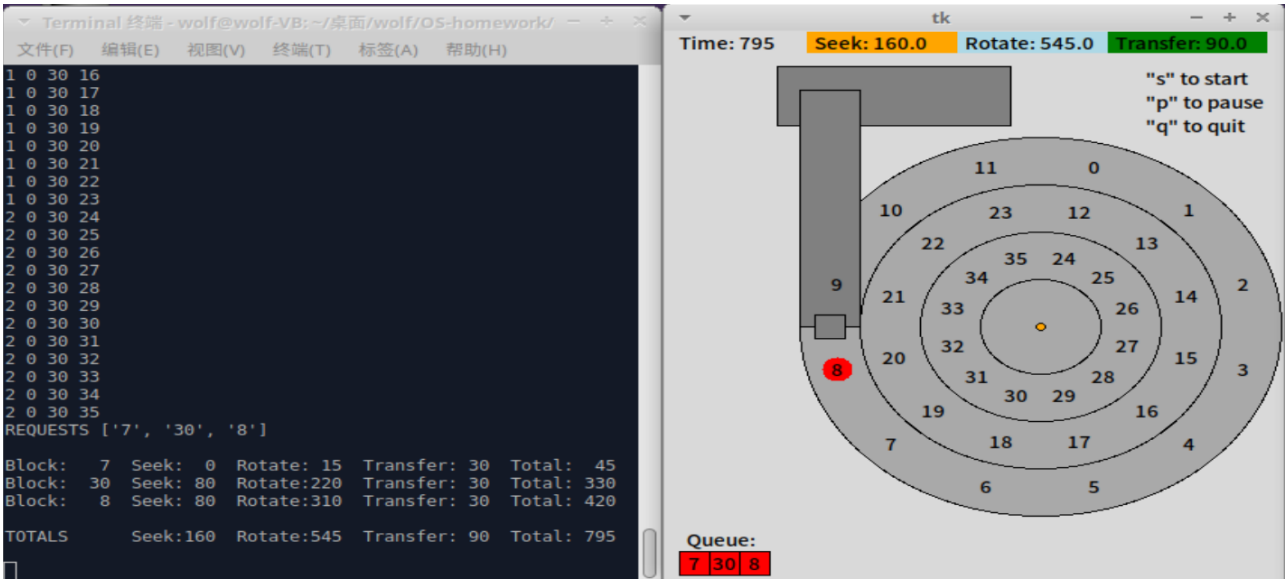
★注意到这次的初始位置发生了变化，如上图。这次我们直接从6开始，而不是0。

★还有一个需要注意的地方，就是多次查询位置的时候要考虑一次结束时应该比开始的时候偏移了30，要把这个减去。

| BLOCK | T寻道 | T旋转 | T传输 | T总 |
|---|---|---|---|---|
| 7 | 0 | 15 | 30 | 45 |
| 30 | 80 | 300-80 | 30 | 330 |
| 8 | 80 | 390-80 | 30 | 420 |

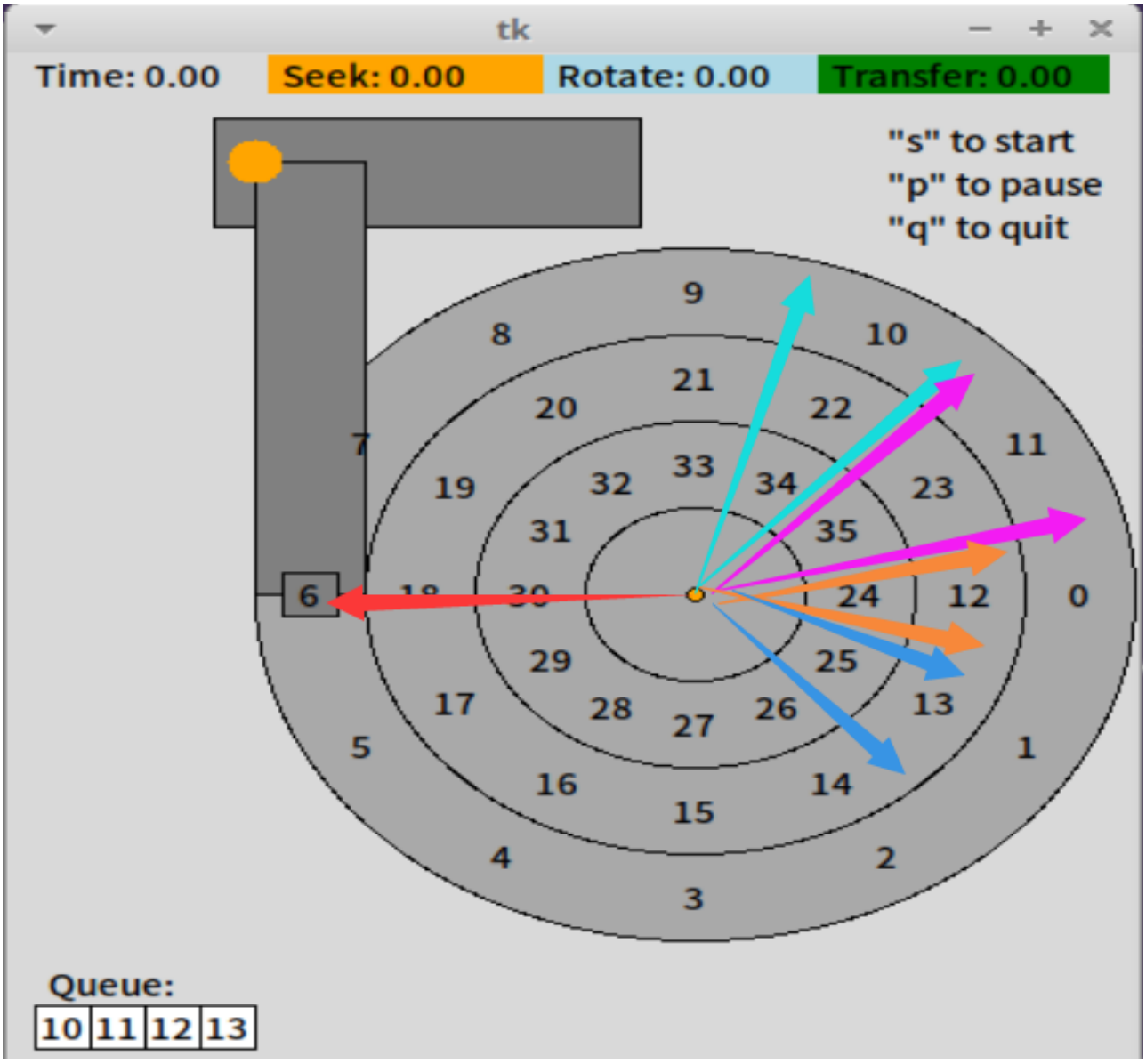
T总=T寻道+T旋转+T传输=795

模拟结果符合预期。




## -a 10，11，12，13

这里模拟的应该是一个顺序读取情况。



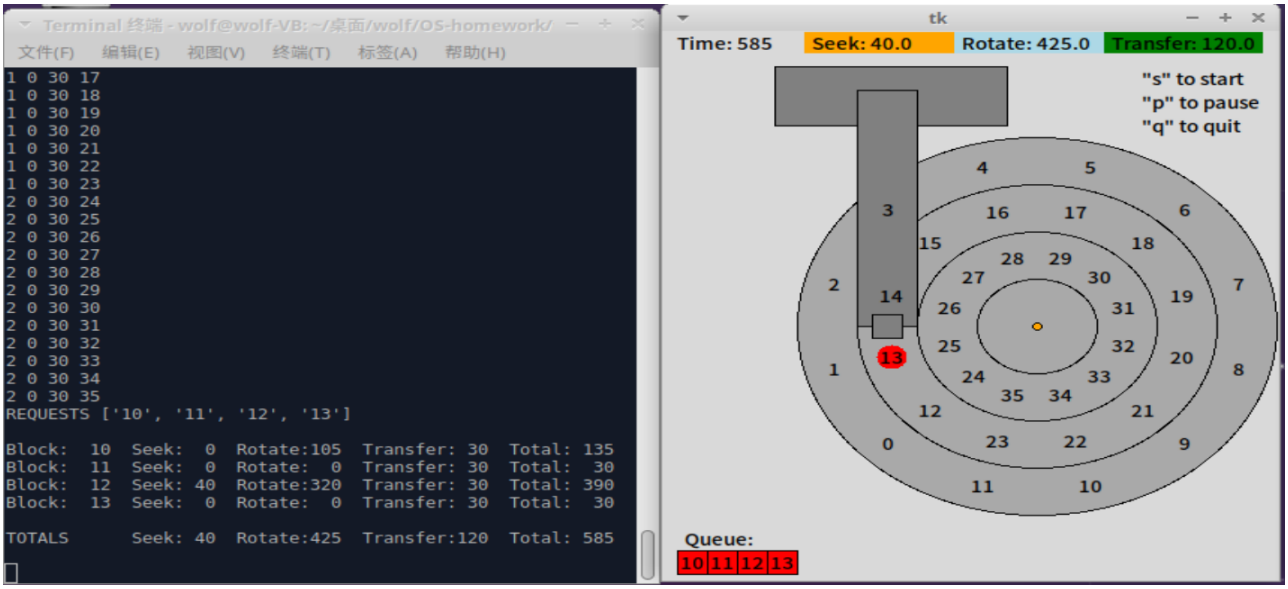
这里顺序读取在10到11和12到13之间没什么特殊的，要注意的是11到12的寻道需要浪费一圈。

同样的，还是列表来做。

| BLOCK | T寻道 | T旋转 | T传输 | T总 |
| --- | --- | --- | --- | --- |
| 10 | 0 | 105 | 30 | 135 |
| 11 | 0 | 0 | 30 | 30 |
| 12 | 80 | 360-80 | 30 | 390 |
| 13 | 0 | 0 | 30 | 30 |

T总=T寻道+T旋转+T传输=585

模拟结果符合预期。




## 37.2

> 执行上述相同请求,但将寻道速率更改为不同值:*-S 2,-S 4,-S 8, -S 10,-S 40, -S 0.1* 时间如何变化?

对于`-a 0`和`-a 6`这两个寻道时间为0的没有影响，不会变化。

★其余的总结来说，有可能因为寻道时间减少而减少多绕一圈的情况从而以360的倍数大大减少总时间；也可能寻道时间减少的不显著而并未跳出旋转时间的“吞并”效应，总时间没有变化；当v减少时，可能会导致寻道时间以很大倍数增大，造成相应的结果。

对于`-a 30`

寻道速率v=2，一次T寻道=40÷2=20。跨两道的寻道时间为40，但由于我们上面解释过，寻道时间可以被视为被吞并，即寻道时间与旋转时间的和保持不变。对于4，8，10，40而言类似，这些的寻道时间变得更短，仍然会被“吞并”。

而对于寻到速率v=0.1，一次T寻道=40÷0.1=400。跨两道的寻道时间为800。此时寻道时间已经大到可“吞并”，必须要单拎出来计算，同时对旋转时间还产生了影响。不变的关系是（寻道时间+旋转时间）=n*360+345，发现n取2符合，即旋转时间为265。

所有速率下的三个时间整理为表格如下：

| 寻道速率**V** | 单次**T**寻道 | **T**寻道 | **T**旋转 | **T**传输 | **T**总 | 比较 |
|---|---|---|---|---|---|---|
| 1（原题） | 40 | 80 | 265 | 30 | 375 | |
| 2 | 20 | 40 | 305 | 30 | 375 | 寻道时间短，不影响总时间 |
| 4 | 10 | 20 | 325 | 30 | 375 | 寻道时间短，不影响总时间 |
| 8 | 5 | 10 | 315 | 30 | 375 | 寻道时间短，不影响总时间 |
| 10 | 4 | 8 | 337 | 30 | 375 | 寻道时间短，不影响总时间 |
| 40 | 1 | 2 | 343 | 30 | 375 | 寻道时间短，不影响总时间 |
| 0.1 | 400 | 800 | 265 | 30 | 1095 | 寻道时间长，不影响总时间 |

对于`-a 7，30，8`

分析同上，但这次需要考虑三次的影响。

所有速率下的三个时间整理为表格如下：

（由于题目程序的模拟问题，在v=0.1时，可能会出现寻道时间1601，旋转时间544的情况，不影响最终结果）

| 寻道速率**V** | 单次**T**寻道 | **T**寻道 | **T**旋转 | **T**传输 | **T**总 | 比较 |
|---|---|---|---|---|---|---|
| 1（原题） | 40 | 0+80+80=160 | 15+220+310=545 | 30+30+30=90 | 795 | |
| 2 | 20 | 0+40+40=80 | 15+260+350=625 | 30+30+30=90 | 795 | 寻道时间短，不影响总时间 |

| 寻道速率**V** | 单次**T**寻道 | **T**寻道 | **T**旋转 | **T**传输 | **T**总 | 比较 |
|---|---|---|---|---|---|---|
| 4 | 10 | 0+20+20=40 | 15+280+10=305 | 30+30+30=90 | 435 | 寻道时间短，由于恰好避免一整圈，显著缩短总时间 |
| 8 | 5 | 0+10+10=20 | 15+290+20=325 | 30+30+30=90 | 435 | 寻道时间短，由于恰好避免一整圈，显著缩短总时间 |
| 10 | 4 | 0+8+8=16 | 15+292+22=329 | 30+30+30=90 | 435 | 寻道时间短，由于恰好避免一整圈，显著缩短总时间 |
| 40 | 1 | 0+2+2=4 | 15+298+28=341 | 30+30+30=90 | 435 | 寻道时间短，由于恰好避免一整圈，显著缩短总时间 |
| 0.1 | 400 | 800+800=1600 | 15+220+310=545 | 30+30+30=90 | 2235 | 寻道时间长，总时间显著变长 |

对于`-a 10，11，12，13`

分析同上，但这次需要考虑三次的影响。

所有速率下的三个时间整理为表格如下：

（由于题目程序的模拟问题，在v=0.1时，可能会出现寻道时间401，旋转时间424的情况，不影响最终结果）

| 寻道速率**V** | 单次**T**寻道 | **T**寻道 | **T**旋转 | **T**传输 | **T**总 | 比较 |
|---|---|---|---|---|---|---|
| 1（原题） | 40 | 0+0+40+0=40 | 105+0+320+0=425 | 30+30+30+30=120 | 585 | |
| 2 | 20 | 0+0+20+0=20 | 105+0+340+0=445 | 30+30+30+30=120 | 585 | 寻道时间短，不影响总时间 |

| 寻道速率**V** | 单次**T**寻道 | **T**寻道 | **T**旋转 | **T**传输 | **T**总 | 比较 |
|---|---|---|---|---|---|---|
| 4 | 10 | 0+0+10+0=10 | 105+0+350+0=455 | 30+30+30+30=120 | 585 | 寻道时间短，不影响总时间 |
| 8 | 5 | 0+0+5+0=5 | 105+0+355+0=460 | 30+30+30+30=120 | 585 | 寻道时间短，不影响总时间 |
| 10 | 4 | 0+0+4+0=4 | 105+0+356+0=461 | 30+30+30+30=120 | 585 | 寻道时间短，不影响总时间 |
| 40 | 1 | 0+0+1+0=1 | 105+0+359+0=464 | 30+30+30+30=120 | 585 | 寻道时间短，不影响总时间 |
| 0.1 | 400 | 0+0+400+0=400 | 105+0+320+0=425 | 30+30+30+30=120 | 945 | 寻道时间长，总时间显著变长 |

## 37.3

> 同样的请求,但改变旋转速率:`-R 0.1,-R 0.5,-R 0.01`。时间如何变化?

★在同条道上的，只需将旋转时间线性增大即可；若涉及不同道上的寻道，可能因为旋转时间相对增大（相当于寻道时间相对减少），从而减少绕一圈，大大减少总时间。

例如`-a 7,30,8`中从30到8的，旋转速度为0.1时，寻道完后还未进入8的扇区，所以只需要转30度，不需要多绕一圈，这一步旋转时间为30/0.1=300比原来的(30+360)/1=390还少。

具体情况如下：

**-a 0：**

访问0号扇区，需要等待磁盘旋转165度。不同旋转速率下，寻道时间均为0，旋转时间和传输时间则根据不同的旋转速率，分别为：

旋转速率0.1：旋转时间1650，传输时间300

旋转速率0.5：旋转时间330，传输时间60

旋转速率0.01：旋转时间16500，传输时间3000

**-a 6：**

与访问0号扇区类似，寻道时间为0，旋转时间发生变化：

旋转速率0.1：旋转时间3450，传输时间300

旋转速率0.5：旋转时间690，传输时间60

旋转速率0.01：旋转时间34500，传输时间3000

**-a 30：**

先进行寻道，寻道时间均为80，然后根据旋转速率，旋转时间与传输时间分别为：

旋转速率0.1：旋转时间3370，传输时间300

旋转速率0.5：旋转时间610，传输时间60

旋转速率0.01：旋转时间34420，传输时间3000

**-a 7,30,8：**

先访问7号扇区，需要旋转15度并等待传输完成，然后移动至内侧，根据此前旋转的时间，计算接下来旋转到30号扇区需要的时间，最后移动回外侧磁道，计算旋转到8号扇区需要的时间。其中寻道时间不随旋转速率变化而变化，均为160，结果如下：

旋转速率0.1：旋转时间3290，传输时间900

旋转速率0.5：旋转时间1250，传输时间180

旋转速率0.01：旋转时间34340，传输时间9000

### -a 10,11,12,13：

访问10,11号扇区后，移动到中间磁道，等待旋转，处理访问12，13扇区的请求。寻道时间均为40，旋转时间和传输时间如下：

旋转速率0.1：旋转时间4610，传输时间1200

旋转速率0.5：旋转时间890，传输时间240

旋转速率0.01：旋转时间46460，传输时间12000

根据以上几组请求的情况，旋转速率的减慢使旋转时间和传输时间大幅增加，尽管因为寻道很快，旋转角度可能小一些，但旋转时间还是很长。

总结

对比图如下

| | **- R 0.1** | **-R 0.5** | **-R 0.01** |
|---|---|---|---|
| -a 0- | 旋转时间增加 | 旋转时间增加 | 旋转时间增加 |
| -a 6 | 旋转时间增加 | 旋转时间增加 | 旋转时间增加 |
| -a 30 | 旋转时间增加 | 旋转时间增加 | 旋转时间增加 |
| -a 7,30,8 | 少旋转一圈，旋转时间减少 | 只是旋转时间增加 | 少旋转一圈，但旋转时间增多 |
| -a 10,11,12,13 | 旋转时间增加 | 旋转时间增加 | 旋转时间增加 |

## 37.4

> 你可能已经注意到,对于一些请求流,一些策略比 *FIFO* 更好。例如,对于请求流 *-a 7,30,8* 处理请求的顺序是什么?现在在相同的工作负载上运行最短寻道时间优先*(SSTF)*调度程序*(-p SSTF)*。每个请求服务需要多长时间*(*寻道、旋转、传输*)?*

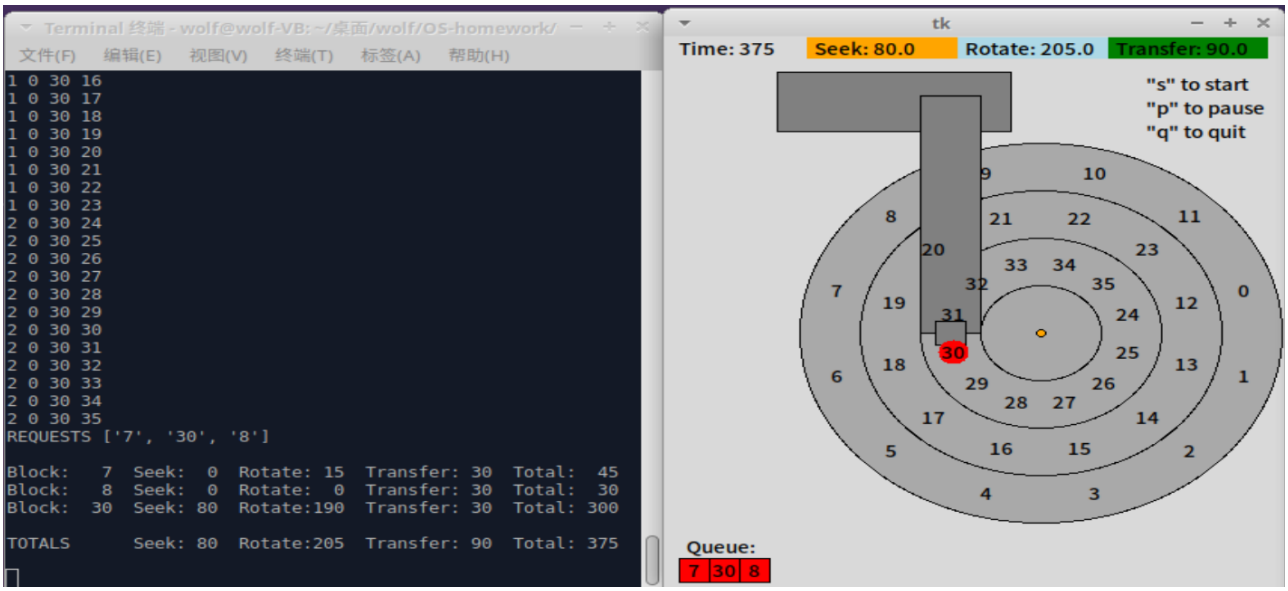
对于请求流-a 7,30,8，FIFO处理请求的顺序为7,30,8，会白白多了一次寻道和一圈的旋转；而最短寻道优先策略SSTF、电梯算法SCAN、最短定位时间优先SPTF的顺序都会是7,8,30。

由第一题我们知道使用后FIFO策略的时间为795

执行如下指令

```
python3 ./disk.py -a 7,30,8 -p SSTF -G
```

结果如下：




可见，最终答案为375，几乎比之前的快了一倍。这就是调度策略契合度的重要性。

## 37.5

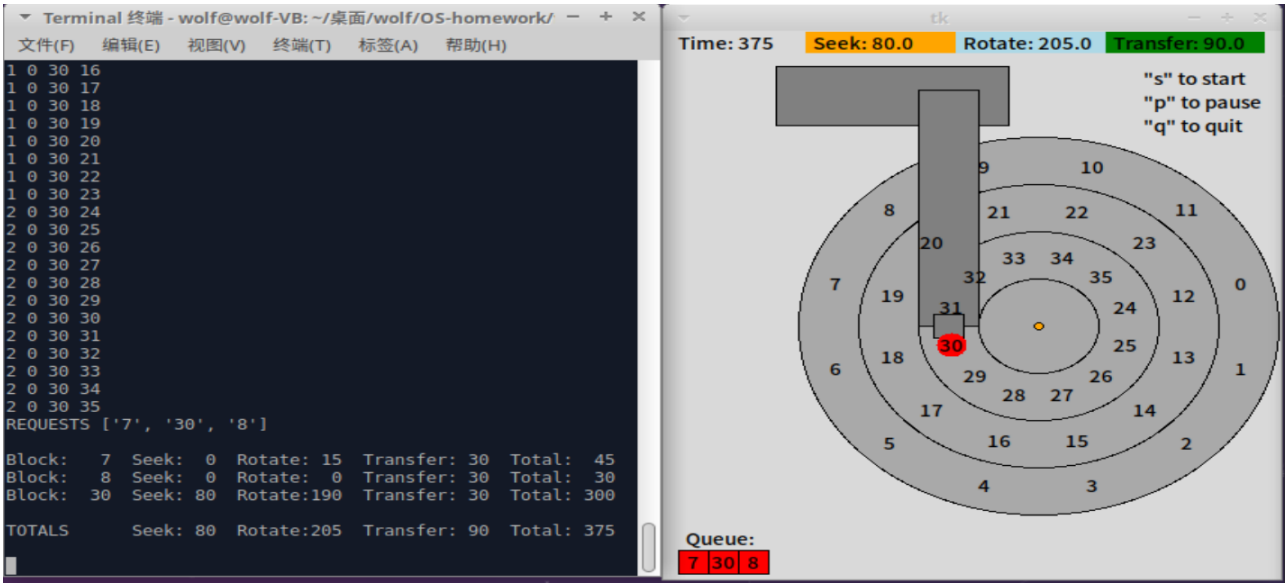
> 现在做同样的事情,但使用最短的访问时间优先*(SATF)*调度程序*(-SATF)*。 它是否对 *-a 7,30,8* 请求有影响？ 找到 *SATF* 明显优于 *SSTF* 的一组请求。出现显著差异的条件是什么*?*

用最短定位时间优先SATF对-a 7,30,8进行处理，无论是从旋转或是寻道的角度讲，顺序都必定是7、8、30是最优的。所以没有改变顺序：

执行如下指令

```
python3 ./disk.py -a 7,30,8 -p SATF -G
```

结果如下：




举个最短定位时间优先比最短寻道时间优先性能优异很多的例子，例如：

7，20，35这个序列。

这个序列是怎么构造出来的呢？

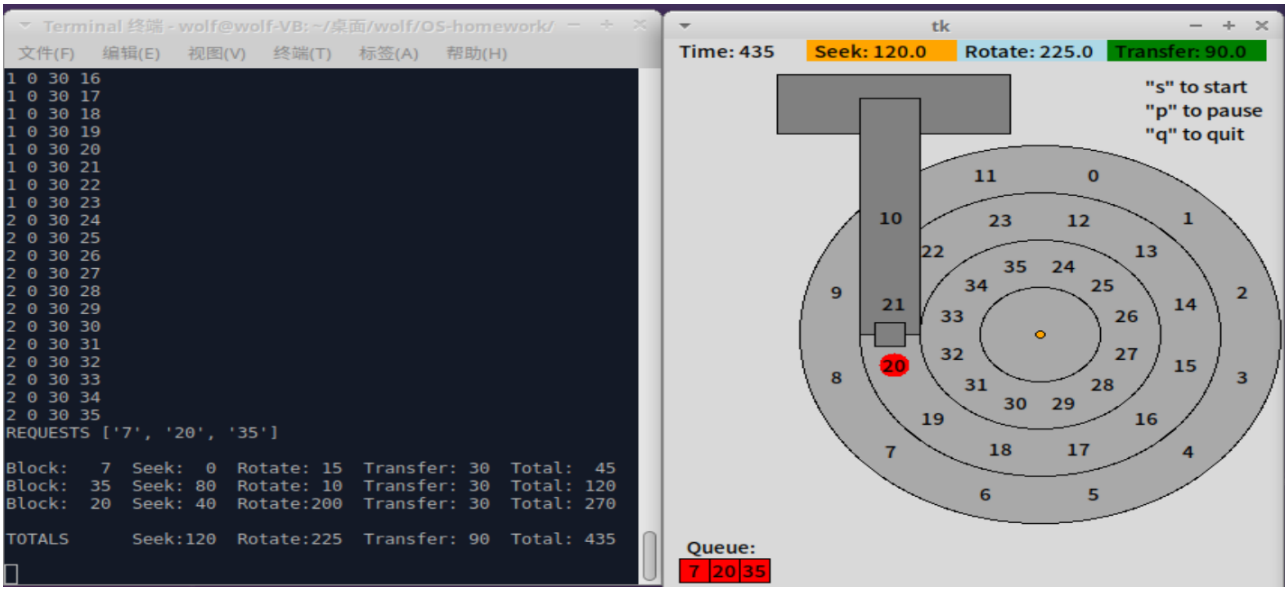
首先我们是从6开始的，我们想抵一个在最外层上读取，所以紧临的7是一个很好的对象，接着如果是SSTF（最短寻道时间优先）的话，肯定要去最邻近道，那我们偏偏让你的代价最大，等你到中间的道上，恰恰好错过20！这意味着你几乎要等整整一圈才能等到20，然后才能去35。而如果是SATF（最短定位时间优先，书上写的是SPTF），此时会去找正好能到的35，完美！

分别执行如下指令

```
python3 ./disk.py -a 7,20,35 -p SATF -G
python3 ./disk.py -a 7,20,35 -p SSTF -G
```

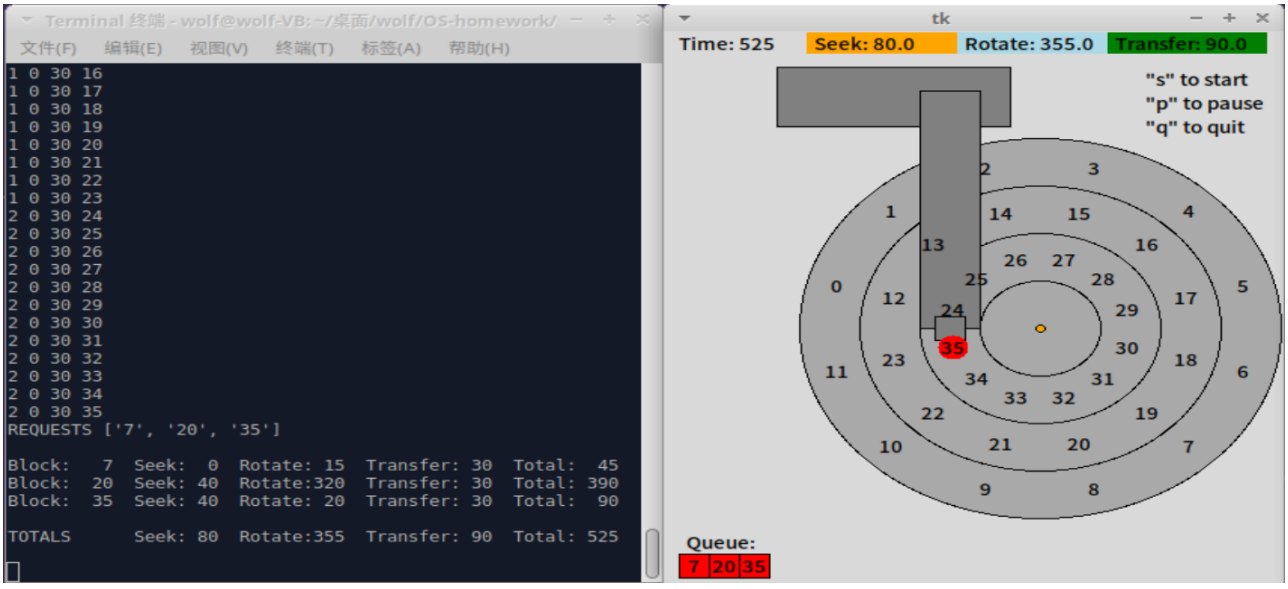
SATF：

7->35->20




SSTF：

7->20->35（恰好落入陷阱里哈哈哈）




这里就可以明显看出最短定位时间在这个序列上的优越性了。

## 37.6

> 你可能已经注意到,该磁盘没有特别好地处理请求流 *-a 10,11,12,13*。这是为什么*?* 你可以引入一个磁道偏斜来解决这个问题*(-o skew,*其中 *skew* 是一个非负整数*)?*考 虑到默认寻道速率,偏斜应该是多少,才能尽量减少这一组请求的总时间*?*对于不同的寻道速率*(*例如,*-S 2,-S 4)*呢*?*一般来说,考虑到寻道速率和扇区布局信息,你能否写出 一个公式来计算偏斜*?*

没有处理好10,11,12,13这一组请求的原因是，12,13号扇区在与10,11号扇区不同的磁道上，而他们在圆周上角度是连续的，处理完10,11号扇区的请求，需要一个寻道时间才能移动到12,13号扇区所在的磁道，而这两个扇区已经在寻道时旋转过去了，因此需要再等待一周的旋转才能处理请求。

引入磁道偏斜，让12,13号扇区在角度上与10,11号扇区不连续，而是偏斜一个角度，寻道时旋转过的小于这个角度，寻道完成后，使12号扇区很快就能转到磁头下进行请求处理，而不需要再旋转接近一圈，就能解决这个问题。默认寻道速率为1，相邻磁道间的移动需要40单位时间，取大于40角度的磁道偏斜，就可以使处理完11号扇区的请求后移动到中间磁道，刚好可以开始处理12号扇区的请求。一个扇区30度，则取2个扇区的磁道偏移，就可以解决问题。设置磁道偏斜为2，运行程序，引入磁道偏斜后，完成寻道时12号扇区还没有旋转过磁头，因此只要等待20的旋转时间，就可以处理12号扇区的请求。最终处理请求的时间为285，比不添加磁道偏斜的时间585少的多。

按照以上的分析，磁道偏斜的角度应该大于寻道时间内旋转过的角度，以减少请求的总时间。假设寻道速率为v，磁道间的距离为s，旋转速率为p，则寻道时间内旋转过的角度为p*(s/v)*。假设一个磁道有*n*个扇区，则一个扇区的角度为*360/n*，偏斜磁道数设为*x*，则有*p*(s/v) < (360/n)*x，x应该取使不等式成立的最小值。

因此可以使用以下公式计算磁道偏斜：

$$x > pns360v(pns/360v)$$

x取使不等式成立的最小整数值

## 第**38**章

运行程序OS-homework/file-raid/raid.py

首先查看README文件，获取一些标志的信息。

- -s 种子seed
- -D 磁盘个数，也就是书上的N
- -C 大块 大小，书上尝试了2，但一般是1
- -n 请求数量
- -S 请求大小
- -W 工作负载，选项为“rand”或者“seq”
- -w 写入占比，100表示全写，0表示全读
- -R 请求的范围
- -L RAID的等级，提供0，1，4，5
- -5 RAID5的左对称（left- symmetric）和左不对称（left-asymmetric）布局，分别为“LS”或"LA"。
- -r 翻转标志

- -t 计算时间
- -c 查看答案

## 38.1

> 使用模拟器执行一些基本的 *RAID* 映射测试。运行不同的级别（*0*、*1*、*4*、*5*），看看你是否可以找出一组请求的映射。 对于 *RAID-5*，看看你是否可以找出左对称（*left- symmetric*）和左不对称（*left-asymmetric*）布局之间的区别。 使用一些不同的随机种子，产生不同于上面的问题。

### RAID0

表 38.1 RAID-0：简单条带化

| 磁盘 0 | 磁盘 1 | 磁盘 2 | 磁盘 3 |
|---|---|---|---|
| 0 | 1 | 2 | 3 |
| 4 | 5 | 6 | 7 |
| 8 | 9 | 10 | 11 |
| 12 | 13 | 14 | 15 |

映射如下：

- 磁盘号 = 地址%磁盘数
- 偏移 = 地址/磁盘数

执行如下指令（与书上给出的简单条带化保持一致）

```
python3 ./raid.py -D 4 -n 5 -L 0 -R 16
```

结果如下：

```
13 1
LOGICAL READ from addr:13 size:4096
  Physical reads/writes?

6 1
LOGICAL READ from addr:6 size:4096
  Physical reads/writes?
```

```
8 1
LOGICAL READ from addr:8 size:4096
  Physical reads/writes?

12 1
LOGICAL READ from addr:12 size:4096
  Physical reads/writes?

7 1
LOGICAL READ from addr:7 size:4096
  Physical reads/writes?
```

根据映射进行计算得结果如下：

```
13 1
LOGICAL READ from addr:13 size:4096
  read  [disk 1, offset 3]

6 1
LOGICAL READ from addr:6 size:4096
  read  [disk 2, offset 1]

8 1
LOGICAL READ from addr:8 size:4096
  read  [disk 0, offset 2]

12 1
LOGICAL READ from addr:12 size:4096
  read  [disk 0, offset 3]

7 1
LOGICAL READ from addr:7 size:4096
  read  [disk 3, offset 1]
```

使用-c验证结果正确。

## RAID1

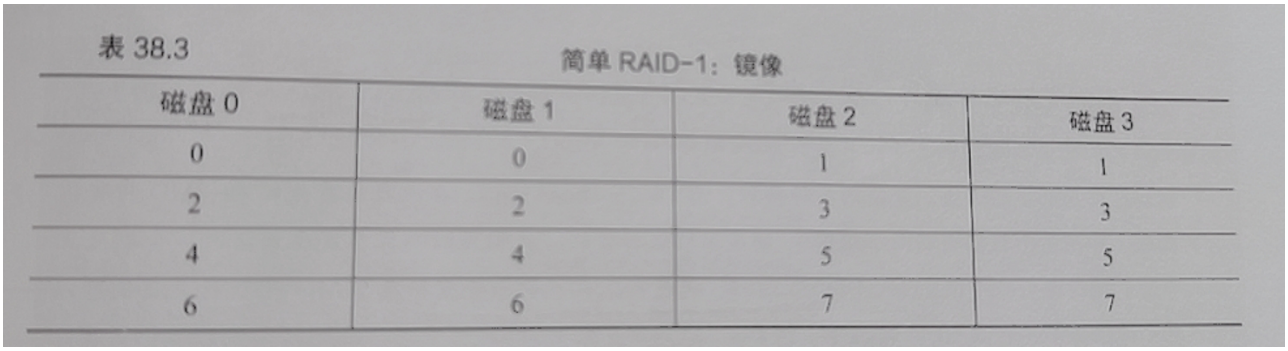


表 38.3 简单 RAID-1：镜像

| 磁盘 0 | 磁盘 1 | 磁盘 2 | 磁盘 3 |
|---|---|---|---|
| 0 | 0 | 1 | 1 |
| 2 | 2 | 3 | 3 |
| 4 | 4 | 5 | 5 |
| 6 | 6 | 7 | 7 |

映射如下：

- 磁盘号 = 2*地址%磁盘数(两个副本都可用)
- 偏移 = 2*地址/磁盘数

执行如下指令（与书上给出的镜像保持一致）

```
python3 ./raid.py -D 4 -n 5 -L 1 -R 8
```

结果如下：

```
6 1
LOGICAL READ from addr:6 size:4096
  Physical reads/writes?

3 1
LOGICAL READ from addr:3 size:4096
  Physical reads/writes?

4 1
LOGICAL READ from addr:4 size:4096
  Physical reads/writes?

6 1
LOGICAL READ from addr:6 size:4096
  Physical reads/writes?

3 1
LOGICAL READ from addr:3 size:4096
  Physical reads/writes?
```

根据映射进行计算得结果如下：

```
6 1
LOGICAL READ from addr:6 size:4096
  read  [disk 1, offset 3]

3 1
LOGICAL READ from addr:3 size:4096
  read  [disk 3, offset 1]

4 1
LOGICAL READ from addr:4 size:4096
  read  [disk 0, offset 2]

6 1
LOGICAL READ from addr:6 size:4096
  read  [disk 1, offset 3]

3 1
LOGICAL READ from addr:3 size:4096
  read  [disk 3, offset 1]
```

使用-c验证结果正确。

## RAID4

表 38.4 具有奇偶校验的 RAID-4

| 磁盘 0 | 磁盘 1 | 磁盘 2 | 磁盘 3 | 磁盘 4 |
|---|---|---|---|---|
| 0 | 1 | 2 | 3 | P0 |
| 4 | 5 | 6 | 7 | P1 |
| 8 | 9 | 10 | 11 | P2 |
| 12 | 13 | 14 | 15 | P3 |

映射如下：

- 磁盘号 = 地址%(磁盘数-1)
- 偏移 = 地址/(磁盘数-1)

执行如下指令（与书上给出的示例保持一致）

```
python3 ./raid.py -D 5 -n 5 -L 4 -R 16
```

结果如下：

```
13 1
LOGICAL READ from addr:13 size:4096
  Physical reads/writes?

6 1
LOGICAL READ from addr:6 size:4096
  Physical reads/writes?

8 1
LOGICAL READ from addr:8 size:4096
  Physical reads/writes?

12 1
LOGICAL READ from addr:12 size:4096
  Physical reads/writes?

7 1
LOGICAL READ from addr:7 size:4096
  Physical reads/writes?
```

根据映射进行计算得结果如下：

```
13 1
LOGICAL READ from addr:13 size:4096
  read  [disk 1, offset 3]
6 1
LOGICAL READ from addr:6 size:4096
  read  [disk 2, offset 1]
8 1
LOGICAL READ from addr:8 size:4096
  read  [disk 0, offset 2]
12 1
LOGICAL READ from addr:12 size:4096
  read  [disk 0, offset 3]
7 1
LOGICAL READ from addr:7 size:4096
  read  [disk 3, offset 1]
```

使用-c验证结果正确。

## RAID5

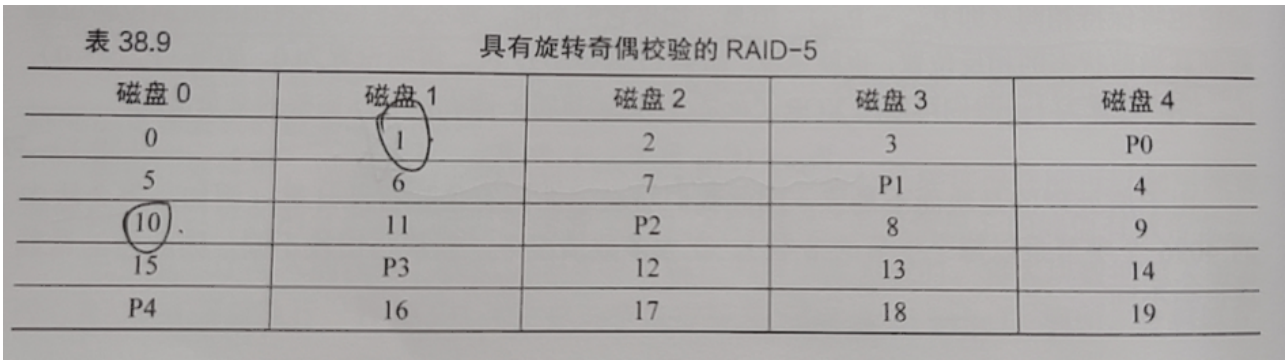


表 38.9 具有旋转奇偶校验的 RAID-5

| 磁盘 0 | 磁盘 1 | 磁盘 2 | 磁盘 3 | 磁盘 4 |
|---|---|---|---|---|
| 0 | 1 | 2 | 3 | P0 |
| 5 | 6 | 7 | P1 | 4 |
| 10 | 11 | P2 | 8 | 9 |
| 15 | P3 | 12 | 13 | 14 |
| P4 | 16 | 17 | 18 | 19 |

分别执行如下指令

```
python3 ./raid.py -D 5 -n 20 -L 5 -R 20 -5 LS -W seq -c
python3 ./raid.py -D 5 -n 20 -L 5 -R 20 -5 LA -W seq -c
```

这相当于是描点作图，取了20个点的位置，可以根据所得结果可以总结出如下：

左对称布局：(与书上是一样的)

| DISK0 | DISK1 | DISK2 | DISK3 | DISK4 |
|---|---|---|---|---|
| 0 | 1 | 2 | 3 | P0 |
| 5 | 6 | 7 | P1 | 4 |
| 10 | 11 | P2 | 8 | 9 |
| 15 | P3 | 12 | 13 | 14 |
| P4 | 16 | 17 | 18 | 19 |

左不对称布局：

| DISK0 | DISK1 | DISK2 | DISK3 | DISK4 |
|---|---|---|---|---|
| 0 | 1 | 2 | 3 | P0 |
| 4 | 5 | 6 | P1 | 7 |
| 8 | 9 | P2 | 10 | 11 |
| 12 | P3 | 13 | 14 | 15 |
| P4 | 16 | 17 | 18 | 19 |

可以比较出RAID5左对称布局和左不对称布局的区别主要在数据块的分布上，左对称分布中， 数据块按照顺序分布在不同磁盘中，下一个数据块放在下一个磁盘上，直到最后一个磁盘。左不对称分布中，如果下一个块磁盘存放了校验块，就跳过下一个磁盘。

## 38.2

> 与第一个问题一样，但这次使用 *-C* 来改变块的大小。大块的大小如何改变映射？

### RAID0

我们还是研究书上给出的示例。

执行如下指令

```
python3 ./raid.py -D 4 -n 16 -L 0 -R 16 -W seq -C 8192 -c
```

根据采样情况可得映射情况如下：

| DISK0 | DISK1 | DISK2 | DISK3 |
| --- | --- | --- | --- |
| 0 | 2 | 4 | 6 |
| 1 | 3 | 5 | 7 |
| 8 | 10 | 12 | 14 |
| 9 | 11 | 13 | 15 |

与书上提供的参考一致。

### RAID1

我们还是研究书上给出的示例。

执行如下指令

```
python3 ./raid.py -D 4 -n 8 -L 1 -R 8 -W seq -C 8192 -c
```

根据采样情况可得映射情况如下：

| DISK0 | DISK1 | DISK2 | DISK3 |
| --- | --- | --- | --- |

| DISK0 | DISK1 | DISK2 | DISK3 |
| --- | --- | --- | --- |
| 0 | 0 | 2 | 2 |
| 1 | 1 | 3 | 3 |
| 4 | 4 | 6 | 6 |
| 5 | 5 | 7 | 7 |

其实也可以理解，这属于是书上没有的内容了。

## RAID4

这次我们换一个问题研究

执行如下指令

```
python3 ./raid.py -D 4 -n 20 -L 4 -R 20 -W seq -C 8192 -c
```

根据采样情况可得映射情况如下：

| DISK0 | DISK1 | DISK2 | DISK3 |
| --- | --- | --- | --- |
| 0 | 2 | 4 | P |
| 1 | 3 | 5 | P |
| 6 | 8 | 10 | P |
| 7 | 9 | 11 | P |

## RAID5

执行如下指令

```
python3 ./raid.py -D 4 -n 20 -L 5 -R 20 -W seq -C 8192 -5 LA -c
```

根据采样情况可得映射情况如下：

| DISK0 | DISK1 | DISK2 | DISK3 |
| --- | --- | --- | --- |
| 0 | 2 | 4 | P |

| DISK0 | DISK1 | DISK2 | DISK3 |
|---|---|---|---|
| 1 | 3 | 5 | P |
| 6 | 8 | P | 10 |
| 7 | 9 | P | 11 |
| 12 | P | 14 | 16 |
| 13 | P | 15 | 17 |

总结

改变大块大小没有改变各级RAID的布局规则，只需要将几个小块看成一个整体，映射情况没有发生改变。

## 38.3

> 执行上述测试，但使用 *r* 标志来反转每个问题的性质。

-r反转后，问题为给出磁盘号和磁盘偏移，计算地址。已知映射关系以及RAID布局的情况下，只需要从布局中找到RAID中某个磁盘特定位置保存的地址，或使用映射公式计算。

直接根据如下不变关系就可得到答案。

- RAID0：地址 = 磁盘数*偏移 + 磁盘号，据此就可以算出1中的地址。
- RAID1：地址 = (磁盘数*偏移 + 磁盘号)/2
- RAID4：地址 = (磁盘数-1)*偏移 + 磁盘号 (可能有1的偏差，因为不确定条带前面是否已有校验块)
- RAID5：可以根据不同的布局直接找到地址。

**RAID0**

表 38.1 RAID-0：简单条带化

| 磁盘 0 | 磁盘 1 | 磁盘 2 | 磁盘 3 |
|---|---|---|---|
| 0 | 1 | 2 | 3 |
| 4 | 5 | 6 | 7 |
| 8 | 9 | 10 | 11 |
| 12 | 13 | 14 | 15 |

映射如下：

- 磁盘号 = 地址%磁盘数
- 偏移 = 地址/磁盘数

执行如下指令

```
python3 ./raid.py -D 4 -n 5 -L 0 -R 16 -r
```

结果如下：

```
13 1
LOGICAL OPERATION is ?
  read  [disk 1, offset 3]

6 1
LOGICAL OPERATION is ?
  read  [disk 2, offset 1]

8 1
LOGICAL OPERATION is ?
  read  [disk 0, offset 2]

12 1
LOGICAL OPERATION is ?
  read  [disk 0, offset 3]

7 1
LOGICAL OPERATION is ?
  read  [disk 3, offset 1]
```

根据映射进行计算得结果如下：

```
13 1
LOGICAL READ from addr:13 size:4096
  read  [disk 1, offset 3]

6 1
LOGICAL READ from addr:6 size:4096
  read  [disk 2, offset 1]

8 1
LOGICAL READ from addr:8 size:4096
```

```
  read  [disk 0, offset 2]

12 1
LOGICAL READ from addr:12 size:4096
  read  [disk 0, offset 3]

7 1
LOGICAL READ from addr:7 size:4096
  read  [disk 3, offset 1]
```

使用-c验证结果正确。

## RAID1

表 38.3 简单 RAID-1：镜像

| 磁盘 0 | 磁盘 1 | 磁盘 2 | 磁盘 3 |
|---|---|---|---|
| 0 | 0 | 1 | 1 |
| 2 | 2 | 3 | 3 |
| 4 | 4 | 5 | 5 |
| 6 | 6 | 7 | 7 |

映射如下：

- 磁盘号 = 2*地址%磁盘数(两个副本都可用)
- 偏移 = 2*地址/磁盘数

执行如下指令（与书上给出的镜像保持一致）

```
python3 ./raid.py -D 4 -n 5 -L 1 -R 8 -r
```

结果如下：

```
6 1
LOGICAL OPERATION is ?
  read  [disk 1, offset 3]

3 1
LOGICAL OPERATION is ?
  read  [disk 3, offset 1]
```

```
4 1
LOGICAL OPERATION is ?
  read  [disk 0, offset 2]

6 1
LOGICAL OPERATION is ?
  read  [disk 1, offset 3]

3 1
LOGICAL OPERATION is ?
  read  [disk 3, offset 1]
```

根据映射进行计算得结果如下：

```
6 1
LOGICAL READ from addr:6 size:4096
  read  [disk 1, offset 3]

3 1
LOGICAL READ from addr:3 size:4096
  read  [disk 3, offset 1]

4 1
LOGICAL READ from addr:4 size:4096
  read  [disk 0, offset 2]

6 1
LOGICAL READ from addr:6 size:4096
  read  [disk 1, offset 3]

3 1
LOGICAL READ from addr:3 size:4096
  read  [disk 3, offset 1]
```

使用-c验证结果正确。

## RAID4

表 38.4 具有奇偶校验的 RAID-4

| 磁盘 0 | 磁盘 1 | 磁盘 2 | 磁盘 3 | 磁盘 4 |
|---|---|---|---|---|
| 0 | 1 | 2 | 3 | P0 |
| 4 | 5 | 6 | 7 | P1 |
| 8 | 9 | 10 | 11 | P2 |
| 12 | 13 | 14 | 15 | P3 |

映射如下：

- 磁盘号 = 地址%(磁盘数-1)
- 偏移 = 地址/(磁盘数-1)

执行如下指令（与书上给出的示例保持一致）

```
python3 ./raid.py -D 5 -n 5 -L 4 -R 16 -r
```

结果如下：

```
13 1
LOGICAL OPERATION is ?
  read  [disk 1, offset 3]
6 1
LOGICAL OPERATION is ?
  read  [disk 2, offset 1]
8 1
LOGICAL OPERATION is ?
  read  [disk 0, offset 2]
12 1
LOGICAL OPERATION is ?
  read  [disk 0, offset 3]
7 1
LOGICAL OPERATION is ?
  read  [disk 3, offset 1]
```

根据映射进行计算得结果如下：

```
13 1
```

```
LOGICAL READ from addr:13 size:4096
  read  [disk 1, offset 3]
6 1
LOGICAL READ from addr:6 size:4096
  read  [disk 2, offset 1]
8 1
LOGICAL READ from addr:8 size:4096
  read  [disk 0, offset 2]
12 1
LOGICAL READ from addr:12 size:4096
  read  [disk 0, offset 3]
7 1
LOGICAL READ from addr:7 size:4096
  read  [disk 3, offset 1]
```

使用-c验证结果正确。

## RAID5

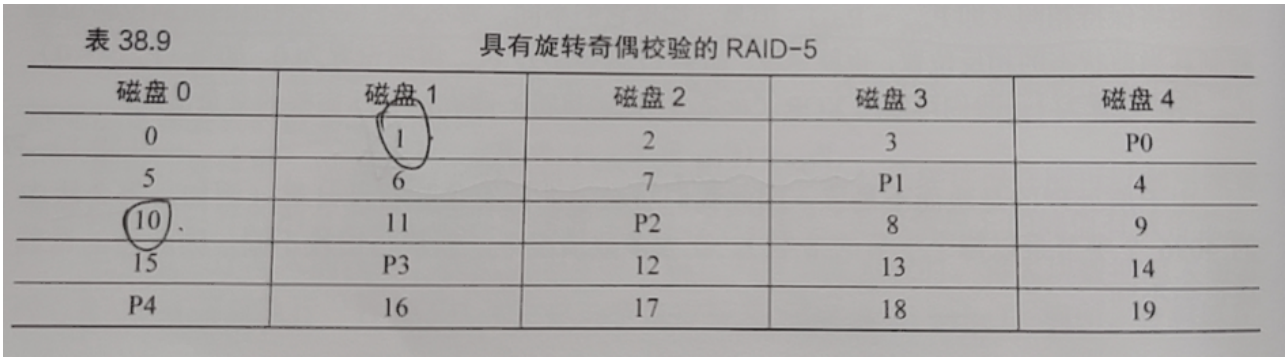


表 38.9 具有旋转奇偶校验的 RAID-5

| 磁盘 0 | 磁盘 1 | 磁盘 2 | 磁盘 3 | 磁盘 4 |
|---|---|---|---|---|
| 0 | 1 | 2 | 3 | P0 |
| 5 | 6 | 7 | P1 | 4 |
| 10 | 11 | P2 | 8 | 9 |
| 15 | P3 | 12 | 13 | 14 |
| P4 | 16 | 17 | 18 | 19 |

分别执行如下指令

```
python3 ./raid.py -D 5 -n 20 -L 5 -R 20 -5 LS -W seq -c
python3 ./raid.py -D 5 -n 20 -L 5 -R 20 -5 LA -W seq -c
```

这相当于是描点作图，取了20个点的位置，可以根据所得结果可以总结出如下：

左对称布局：(与书上是一样的)

| DISK0 | DISK1 | DISK2 | DISK3 | DISK4 |
|---|---|---|---|---|
| 0 | 1 | 2 | 3 | P0 |
| 5 | 6 | 7 | P1 | 4 |

| DISK0 | DISK1 | DISK2 | DISK3 | DISK4 |
| --- | --- | --- | --- | --- |
| 10 | 11 | P2 | 8 | 9 |
| 15 | P3 | 12 | 13 | 14 |
| P4 | 16 | 17 | 18 | 19 |

左不对称布局：

| DISK0 | DISK1 | DISK2 | DISK3 | DISK4 |
| --- | --- | --- | --- | --- |
| 0 | 1 | 2 | 3 | P0 |
| 4 | 5 | 6 | P1 | 7 |
| 8 | 9 | P2 | 10 | 11 |
| 12 | P3 | 13 | 14 | 15 |
| P4 | 16 | 17 | 18 | 19 |

按照对应映射公式可以得到答案。

## 38.4

> 现在使用反转标志，但用*-S* 标志增加每个请求的大小。尝试指定 *8 KB*、*12 KB* 和 *16 KB* 的大小，同时改变 *RAID* 级别。 当请求的大小增加时，底层 *IO* 模式会发生什么？请务必在顺序工作负载上尝试此操作（*-W sequential*）。 对于什么请求大小，*RAID-4* 和 *RAID-5* 的 *I0* 效率更高？

本题要求改变请求的大小，并使用-r反转，即观察不同大小的请求磁盘I/O的情况以及不同RAID级别下磁盘I/O的情况。

### -S 8K

**RAID0：**

分别执行如下指令，观察读写请求

```
python3 ./raid.py -n 5 -L 0 -R 20 -r -S 8K -W seq
python3 ./raid.py -n 5 -L 0 -R 20 -r -S 8K -W seq -w 100
```

以读为例，如下：

```
```

```
0 2
LOGICAL OPERATION is ?
  read  [disk 0, offset 0]
  read  [disk 1, offset 0]

2 2
LOGICAL OPERATION is ?
  read  [disk 2, offset 0]
  read  [disk 3, offset 0]

4 2
LOGICAL OPERATION is ?
  read  [disk 0, offset 1]
  read  [disk 1, offset 1]

6 2
LOGICAL OPERATION is ?
  read  [disk 2, offset 1]
  read  [disk 3, offset 1]

8 2
LOGICAL OPERATION is ?
  read  [disk 0, offset 2]
  read  [disk 1, offset 2]
```

由于一次只能执行一个读操作，显然需要两次完成。

**RAID1：**

分别执行如下指令，观察读写请求

```
python3 ./raid.py -n 5 -L 1 -R 20 -r -S 8K -W seq
python3 ./raid.py -n 5 -L 1 -R 20 -r -S 8K -W seq -w 100
```

以写为例，如下：

```
0 2
LOGICAL OPERATION is ?
  write [disk 0, offset 0]    write [disk 1, offset 0]
  write [disk 2, offset 0]    write [disk 3, offset 0]
```

```
2 2
LOGICAL OPERATION is ?
  write [disk 0, offset 1]    write [disk 1, offset 1]
  write [disk 2, offset 1]    write [disk 3, offset 1]

4 2
LOGICAL OPERATION is ?
  write [disk 0, offset 2]    write [disk 1, offset 2]
  write [disk 2, offset 2]    write [disk 3, offset 2]

6 2
LOGICAL OPERATION is ?
  write [disk 0, offset 3]    write [disk 1, offset 3]
  write [disk 2, offset 3]    write [disk 3, offset 3]

8 2
LOGICAL OPERATION is ?
  write [disk 0, offset 4]    write [disk 1, offset 4]
  write [disk 2, offset 4]    write [disk 3, offset 4]
```

由于有镜像的存在，每次写入两个块，同时考虑镜像块，故有4次操作。

**RAID4：**

分别执行如下指令，观察读写请求

```
python3 ./raid.py -n 5 -L 4 -R 20 -r -S 8K -W seq
python3 ./raid.py -n 5 -L 4 -R 20 -r -S 8K -W seq -w 100
```

以写为例，如下：

```
0 2
LOGICAL OPERATION is ?
  read  [disk 2, offset 0]
  write [disk 0, offset 0]    write [disk 1, offset 0]    write
[disk 3, offset 0]

2 2
LOGICAL OPERATION is ?
  read  [disk 2, offset 0]    read  [disk 3, offset 0]
  write [disk 2, offset 0]    write [disk 3, offset 0]
```

```
  read  [disk 0, offset 1]    read  [disk 3, offset 1]
  write [disk 0, offset 1]    write [disk 3, offset 1]

4 2
LOGICAL OPERATION is ?
  read  [disk 0, offset 1]
  write [disk 1, offset 1]    write [disk 2, offset 1]    write
[disk 3, offset 1]

6 2
LOGICAL OPERATION is ?
  read  [disk 2, offset 2]
  write [disk 0, offset 2]    write [disk 1, offset 2]    write
[disk 3, offset 2]

8 2
LOGICAL OPERATION is ?
  read  [disk 2, offset 2]    read  [disk 3, offset 2]
  write [disk 2, offset 2]    write [disk 3, offset 2]
  read  [disk 0, offset 3]    read  [disk 3, offset 3]
  write [disk 0, offset 3]    write [disk 3, offset 3]
```

正常情况下写入一个块，应该要先读地址对应的块，再读奇偶校验块，判断是否需要改变，然后再写入地址对应的块和校验块即可。

请求大小为8k的话，其实是写入地址和地址+1的块。

若两个块不连续，则需要两次上述的操作，需要8个I/O操作；

若这两个块是连续的（偏移量一样），奇妙的事情发生了，此时由于这两个块共用同一个校验位，故只需要写入这两个快和校验位P，由原来的8个I/O操作减少到4个。

**RAID5：**

分别执行如下指令，观察读写请求

```
python3 ./raid.py -n 5 -L 5 -R 20 -r -S 8K -W seq
python3 ./raid.py -n 5 -L 5 -R 20 -r -S 8K -W seq -w 100
```

以写为例，如下：

```
0 2
LOGICAL OPERATION is ?
  read  [disk 2, offset 0]
  write [disk 0, offset 0]    write [disk 1, offset 0]    write
[disk 3, offset 0]

2 2
LOGICAL OPERATION is ?
  read  [disk 2, offset 0]    read  [disk 3, offset 0]
  write [disk 2, offset 0]    write [disk 3, offset 0]
  read  [disk 3, offset 1]    read  [disk 2, offset 1]
  write [disk 3, offset 1]    write [disk 2, offset 1]

4 2
LOGICAL OPERATION is ?
  read  [disk 3, offset 1]
  write [disk 0, offset 1]    write [disk 1, offset 1]    write
[disk 2, offset 1]

6 2
LOGICAL OPERATION is ?
  read  [disk 0, offset 2]
  write [disk 2, offset 2]    write [disk 3, offset 2]    write
[disk 1, offset 2]

8 2
LOGICAL OPERATION is ?
  read  [disk 0, offset 2]    read  [disk 1, offset 2]
  write [disk 0, offset 2]    write [disk 1, offset 2]
  read  [disk 1, offset 3]    read  [disk 0, offset 3]
  write [disk 1, offset 3]    write [disk 0, offset 3]
```

RAID5与RAID4基本相似。

### -S 12K

分别执行如下指令完成测试

```
python3 ./raid.py -n 5 -L 0 -R 20 -r -S 12K -W seq
python3 ./raid.py -n 5 -L 0 -R 20 -r -S 12K -W seq -w 100
python3 ./raid.py -n 5 -L 1 -R 20 -r -S 12K -W seq
python3 ./raid.py -n 5 -L 1 -R 20 -r -S 12K -W seq -w 100
python3 ./raid.py -n 5 -L 4 -R 20 -r -S 12K -W seq
python3 ./raid.py -n 5 -L 4 -R 20 -r -S 12K -W seq -w 100
python3 ./raid.py -n 5 -L 5 -R 20 -r -S 12K -W seq
python3 ./raid.py -n 5 -L 5 -R 20 -r -S 12K -W seq -w 100
```

篇幅所限，测试结果不再赘述。总结如下：

请求大小修改为12K，对于RAID0与RAID1，只是需要多对一个块进行处理。因此与8K类似，RAID0需要3次I/O完成请求，RAID1需要3次读操作完成读请求，6次写操作完成写请求。

RAID4的随机读和顺序读也与8K类似，需要3次读完成。随机写有所不同，如果在同一个条带上进行写，那只需要三个块进行异或，然后一次将包括校验块在内的四个块全部写入，故需要4次写。如果有2个块在同一条带上，那么这两个块的写入可以采取加法奇偶校验(4次写操作)，另一个单独在其他条带的块不论采用哪种方式，都需要4次写操作，故一共8次写操作。

RAID4顺序写时，写操作数明显减少了，因为每次请求都是对一个条带上的三个块进行写请求，可以采用全条带写入，即直接将三个块异或，然后全部和奇偶校验块一起写入。

RAID5的情况与RAID4是基本相同。

### -S 16K

分别执行如下指令完成测试

```
python3 ./raid.py -n 5 -L 0 -R 20 -r -S 16K -W seq
python3 ./raid.py -n 5 -L 0 -R 20 -r -S 16K -W seq -w 100
python3 ./raid.py -n 5 -L 1 -R 20 -r -S 16K -W seq
python3 ./raid.py -n 5 -L 1 -R 20 -r -S 16K -W seq -w 100
python3 ./raid.py -n 5 -L 4 -R 20 -r -S 16K -W seq
python3 ./raid.py -n 5 -L 4 -R 20 -r -S 16K -W seq -w 100
python3 ./raid.py -n 5 -L 5 -R 20 -r -S 16K -W seq
python3 ./raid.py -n 5 -L 5 -R 20 -r -S 16K -W seq -w 100
```

篇幅所限，测试结果不再赘述。总结如下：

对于16K的请求，RAID0与RAID1的情况没有发生变化，只是需要多处理一个块。RAID0完成请求需要4次I/O。RAID1完成读需要4次读操作，完成写需要8次写操作。

RAID4的随机读和顺序读4次读操作完成。随机写时，有以下2种情况：

情况1：一个请求分布在两个条带上，两个条带上的块数分别为3,1，

情况2：一个请求分布在两个条带上，两个条带上的块数分别为2,2。

考虑3,1的情况，3个块在同一个条带上可以使用全条带写入(4次写)，剩下一个块4次写单独处理，共8次写。另一种2,2的情况，每一个条带上采用加法奇偶校验，各需要4次写，故也需要8次写。

对于顺序写，情况是与随机写相同的，因为请求大小比1个条带的数据块要多。因此顺序写也是以上的两种模式。

RAID5与RAID4基本相同。

### 总结

综合以上的所有分析，对于4个磁盘的情况下，请求块数越接近(小于等于)一个条带的块数，RAID4和RAID5的写性能更好。即RAID4/5更适合接近一个条带块数的顺序写入。因为在这种情况下，加法奇偶校验可以比减法奇偶校验使用更少的写操作完成请求，最好的情况下，可以使用全条带写入直接完成写入，而不需要读取数据块。

## 38.5

> 使用模拟器的定时模式（*-t*）来估计 *100* 次随机读取到 *RAID* 的性能，同时改变 *RAID* 级别，使用 *4* 个磁盘。

具体性能书上有详细的表格：

表 38.10 RAID 容量、可靠性和性能

| | RAID-0 | RAID-1 | RAID-4 | RAID-5 |
|---|---|---|---|---|
| 容量 | $N$ | $N/2$ | $N-1$ | $N-1$ |
| 可靠性 | 0 | 1（肯定） | | |
| | | $N/2$（如果走运） | | |
| 吞吐量 | | | | |
| 顺序读 | $N \cdot S$ | $(N/2) \cdot S$ | $(N-1) \cdot S$ | $(N-1) \cdot S$ |
| 顺序写 | $N \cdot S$ | $(N/2) \cdot S$ | $(N-1) \cdot S$ | $(N-1) \cdot S$ |
| 随机读 | $N \cdot R$ | $N \cdot R$ | $(N-1) \cdot R$ | $N \cdot R$ |
| 随机写 | $N \cdot R$ | $(N/2) \cdot R$ | $1/2 \cdot R$ | $N/4 \cdot R$ |

下面分别执行指令进行模拟

**RAID0：**

```
python3 ./raid.py -L 0 -t -n 100 -c
```

结果如下：

```
disk:0  busy: 100.00  I/Os:    28 (sequential:0 nearly:1 random:27)
disk:1  busy:  93.91  I/Os:    29 (sequential:0 nearly:6 random:23)
disk:2  busy:  87.92  I/Os:    24 (sequential:0 nearly:0 random:24)
disk:3  busy:  65.94  I/Os:    19 (sequential:0 nearly:1 random:18)

STAT totalTime 275.69999999999993
```

**RAID1：**

```
python3 ./raid.py -L 1 -t -n 100 -c
```

结果如下：

```
disk:0  busy: 100.00  I/Os:    28 (sequential:0 nearly:1 random:27)
disk:1  busy:  86.98  I/Os:    24 (sequential:0 nearly:0 random:24)
disk:2  busy:  97.52  I/Os:    29 (sequential:0 nearly:3 random:26)
disk:3  busy:  65.23  I/Os:    19 (sequential:0 nearly:1 random:18)

STAT totalTime 278.7
```

**RAID4：**

```
python3 ./raid.py -L 4 -t -n 100 -c
```

结果如下：

```
disk:0  busy:  78.48  I/Os:    30 (sequential:0 nearly:0 random:30)
disk:1  busy: 100.00  I/Os:    40 (sequential:0 nearly:3 random:37)
disk:2  busy:  76.46  I/Os:    30 (sequential:0 nearly:2 random:28)
disk:3  busy:   0.00  I/Os:     0 (sequential:0 nearly:0 random:0)

STAT totalTime 386.1000000000002
```

**RAID5：**

```
python3 ./raid.py -L 5 -t -n 100 -c
```

结果如下：

```
disk:0  busy: 100.00  I/Os:    28 (sequential:0 nearly:1 random:27)
disk:1  busy:  95.84  I/Os:    29 (sequential:0 nearly:5 random:24)
disk:2  busy:  87.60  I/Os:    24 (sequential:0 nearly:0 random:24)
disk:3  busy:  65.70  I/Os:    19 (sequential:0 nearly:1 random:18)

STAT totalTime 276.7
```

总结：

随机读写、顺序读写均与教材表格总结符合得较好。

## 第40章

运行程序OS-homework/file-implementation/vsfs.py

首先阅读README文件，获得一些标志的含义

- -s 种子seed
- -i inode数量

- -d 数据块数量
- -n 请求数量
- -r 打印设置
- -p 打印最终文件系统架构
- -c 计算答案

此外，README中还有操作的影响和inode的结构等。

## 40.1

> 用一些不同的随机种子（比如 *17*、*18*、*19*、*20*）运行模拟器，看看你是否能确定每次状态变化之间一定发生了哪些操作。

### 前置知识：不同操作的作用与对磁盘的影响

- mkdir() - 创建文件夹：修改inode位图，增加一个inode用来存放新目录元数据，向存放新目录的目录块中增加一个条目，修改data位图，增加一个数据块用于存放新目录的内容，更新相应inode中的引用计数
- creat() - 创建新的空文件：修改inode位图，增加一个inode用来存放新文件元数据，向存放新文件的目录块中增加一个条目，更新相应inode中的引用计数
- open(), write() ,close() - 将一个块添加到文件：修改data位图，增加一个数据块用于存放文件的新内容，修改inode中的数据块地址字段
- link() - 创建一个文件的硬链接：修改inode，增加其中的引用计数，在保存链接的目录块中增加一个条目
- unlink() - 删除一个硬链接 (如果 linkcnt==0,删除文件)：修改inode，减小其中的引用计数，在保存链接的目录块中删除一个条目，当引用计数减为0时，删除文件，释放inode、数据块，修改inode位图、data位图

总结来说：

- 创建文件夹：修改inode位图和data位图。文件夹被创建时自带一个data块来记录子目录信息。
- 创建文件：只修改inode位图。文件只需要被记录。

### 前置知识：**inode**的结构

每个inode都有三个字段：

- 第一个字段指示文件的类型（例如，f表示常规文件，d表示目录）；

- 第二个表示数据块属于一个文件（在这里，文件只能为空，这将数据块的地址设置为-1，或者大小为一个块，这将具有非负数地址）；
- 第三个显示文件的引用计数或目录。

下面是一个范例

> 例如，下面的inode是一个常规文件，它是空（地址字段设置为-1），并且在文件系统中只有一个链接：
> [f a:-1 r:1]
> 如果同一个文件分配了一个块（比如块10），则会显示如下所示：
> [f a:10 r:1]
> 如果有人创建了一个指向该索引节点的硬链接，它就会变成：
> [f a:10 r:2]
> 最后，数据块可以保留用户数据或目录数据。如果已填充对于目录数据块内的每个条目的形式（name，inumber），其中“name”是文件或目录的名称“inumber”是文件的索引节点编号。因此一个空的根目录看起来是这样的，假设根索引节点是0:
> [（.，0）（..，0）]
> 如果我们在根目录中添加一个文件“f”，它已经被分配inode编号1，则根目录内容将变为：
> [（.，0）（..0）（f，1）]
> 如果数据块包含用户数据，则仅显示为单个字符在块内，例如“h”。如果它是空的并且未分配，那么只需要一对显示了空括号[]。
> 因此，整个文件系统如下所示：

```
inode bitmap 11110000
inodes       [d a:0 r:6] [f a:1 r:1] [f a:-1 r:1] [d a:2 r:2] []
...
data bitmap  11100000
data         [(.,0) (..,0) (y,1) (z,2) (f,3)] [u] [(.,3) (..,0)] []
...
```

以上前置知识都可以在README中查询到。

## seed17

执行如下指令

```
python3 ./vsfs.py -n 6 -s 17 -c
```

结果如下：

```
Initial state
```

```
inode bitmap  10000000
inodes        [d a:0 r:2][][][][][][][]
data bitmap   10000000
data          [(.,0) (..,0)][][][][][][][]

mkdir("/u");

inode bitmap  11000000
inodes        [d a:0 r:3][d a:1 r:2][][][][][][]
data bitmap   11000000
data          [(.,0) (..,0) (u,1)][(.,1) (..,0)][][][][][][]

creat("/a");

inode bitmap  11100000
inodes        [d a:0 r:3][d a:1 r:2][f a:-1 r:1][][][][][]
data bitmap   11000000
data          [(.,0) (..,0) (u,1) (a,2)][(.,1) (..,0)][][][][][][]

unlink("/a");

inode bitmap  11000000
inodes        [d a:0 r:3][d a:1 r:2][][][][][][]
data bitmap   11000000
data          [(.,0) (..,0) (u,1)][(.,1) (..,0)][][][][][][]

mkdir("/z");

inode bitmap  11100000
inodes        [d a:0 r:4][d a:1 r:2][d a:2 r:2][][][][][]
data bitmap   11100000
data          [(.,0) (..,0) (u,1) (z,2)][(.,1) (..,0)][(.,2) (..,0)]
[][][][][][]

mkdir("/s");

inode bitmap  11110000
inodes        [d a:0 r:5][d a:1 r:2][d a:2 r:2][d a:3 r:2][][][][][]
data bitmap   11110000
data          [(.,0) (..,0) (u,1) (z,2) (s,3)][(.,1) (..,0)][(.,2)
(..,0)][(.,3) (..,0)][][][][][]
```

```
creat("/z/x");

inode bitmap  11111000
inodes        [d a:0 r:5][d a:1 r:2][d a:2 r:2][d a:3 r:2][f a:-1
r:1][][][]
data bitmap   11110000
data          [(.,0) (..,0) (u,1) (z,2) (s,3)][(.,1) (..,0)][(.,2)
(..,0) (x,4)][(.,3) (..,0)][][][][]
```

解释如下：

- 操作1同时修改了inode位图和data位图，只有mkdir()可以做到。查看1号inode,发现新建了一个目录，其数据存放在1号数据块，在0号数据块中查看新增加的条目，指示新建的目录名为“u”，所以操作1是mkdir(“/u”)
- 操作2只修改了inode位图，所以是creat()。查看2号inode,发现新建了一个文件，在0号数据块中查看新增加的条目，指示新建的文件名为“a”，所以操作2是creat(“/a”)
- 操作3修改了inode位图，删除了inode和目录块中的条目，所以是unlink()。发现删除的是2号inode，所以操作3是unlink(“/a”)
- 操作4同时修改了inode位图和data位图，只有mkdir()可以做到。查看2号inode,发现新建了一个目录，其数据存放在2号数据块，在0号数据块中查看新增加的条目，指示新建的目录名为“z”，所以操作4是mkdir(“/z”)
- 操作5同时修改了inode位图和data位图，只有mkdir()可以做到。查看3号inode,发现新建了一个目录，其数据存放在3号数据块，在0号数据块中查看新增加的条目，指示新建的目录名为“s”，所以操作5是mkdir(“/s”)
- 操作6只修改了inode位图，所以是creat()。查看4号inode,发现新建了一个文件，在3号数据块（目录z的目录块）中查看新增加的条目，指示新建的文件名为“x”，所以操作6是creat(“/z/x”)

## seed 18

执行如下指令

```
python3 ./vsfs.py -n 6 -s 18 -c
```

结果如下：

```
Initial state
```

```
inode bitmap  10000000
inodes        [d a:0 r:2][][][][][][][]
data bitmap   10000000
data          [(.,0) (..,0)][][][][][][][]

mkdir("/f");

inode bitmap  11000000
inodes        [d a:0 r:3][d a:1 r:2][][][][][][]
data bitmap   11000000
data          [(.,0) (..,0) (f,1)][(.,1) (..,0)][][][][][][]

creat("/s");

inode bitmap  11100000
inodes        [d a:0 r:3][d a:1 r:2][f a:-1 r:1][][][][][]
data bitmap   11000000
data          [(.,0) (..,0) (f,1) (s,2)][(.,1) (..,0)][][][][][][]

mkdir("/h");

inode bitmap  11110000
inodes        [d a:0 r:4][d a:1 r:2][f a:-1 r:1][d a:2 r:2][][][][]
data bitmap   11100000
data          [(.,0) (..,0) (f,1) (s,2) (h,3)][(.,1) (..,0)][(.,3)
(..,0)][][][][][]

fd=open("/s", O_WRONLY|O_APPEND); write(fd, buf, BLOCKSIZE);
close(fd);

inode bitmap  11110000
inodes        [d a:0 r:4][d a:1 r:2][f a:3 r:1][d a:2 r:2][][][][]
data bitmap   11110000
data          [(.,0) (..,0) (f,1) (s,2) (h,3)][(.,1) (..,0)][(.,3)
(..,0)][f][][][][]

creat("/f/o");

inode bitmap  11111000
inodes        [d a:0 r:4][d a:1 r:2][f a:3 r:1][d a:2 r:2][f a:-1
r:1][][][][]
data bitmap   11110000
```

```
data          [(.,0) (..,0) (f,1) (s,2) (h,3)][(.,1) (..,0) (o,4)]
[(.,3) (..,0)][f][][][][]

creat("/c");

inode bitmap  11111100
inodes        [d a:0 r:4][d a:1 r:2][f a:3 r:1][d a:2 r:2][f a:-1
r:1][f a:-1 r:1][][]
data bitmap   11110000
data          [(.,0) (..,0) (f,1) (s,2) (h,3) (c,5)][(.,1) (..,0)
(o,4)][(.,3) (..,0)][f][][][][]
```

解释如下：

- 操作1同时修改了inode位图和data位图，只有mkdir()可以做到。查看1号inode,发现新建了一个目录，其数据存放在1号数据块，在0号数据块中查看新增加的条目，指示新建的目录名为“f”，所以操作1是mkdir(“/f”)
- 操作2只修改了inode位图，所以是creat()。查看2号inode,发现新建了一个文件，在0号数据块中查看新增加的条目，指示新建的文件名为“s”，所以操作2是creat(“/s”)
- 操作3同时修改了inode位图和data位图，只有mkdir()可以做到。查看3号inode,发现新建了一个目录，其数据存放在2号数据块，在0号数据块中查看新增加的条目，指示新建的目录名为“h”，所以操作3是mkdir(“/h”)
- 操作4修改了data位图，修改了2号inode（文件s）中的地址字段，增加了3号数据块，所以操作4是fd=open(“/s”,
- O_WRONLY|O_APPEND); write(fd, buf, BLOCKSIZE); close(fd)
- 操作5只修改了inode位图，所以是creat()。查看4号inode,发现新建了一个文件，在1号数据块（目录f的目录块）中查看新增加的条目，指示新建的文件名为“o”，所以操作5是creat(“/f/o”)
- 操作6只修改了inode位图，所以是creat()。查看5号inode,发现新建了一个文件，在0号数据块中查看新增加的条目，指示新建的文件名为“c”，所以操作6是creat(“/c”)

## seed 19

执行如下指令

```
python3 ./vsfs.py -n 6 -s 19 -c
```

结果如下：

```
Initial state

inode bitmap  10000000
inodes        [d a:0 r:2][][][][][][][]
data bitmap   10000000
data          [(.,0) (..,0)][][][][][][][]

creat("/k");

inode bitmap  11000000
inodes        [d a:0 r:2][f a:-1 r:1][][][][][][]
data bitmap   10000000
data          [(.,0) (..,0) (k,1)][][][][][][][]

creat("/g");

inode bitmap  11100000
inodes        [d a:0 r:2][f a:-1 r:1][f a:-1 r:1][][][][][]
data bitmap   10000000
data          [(.,0) (..,0) (k,1) (g,2)][][][][][][][]

fd=open("/k", O_WRONLY|O_APPEND); write(fd, buf, BLOCKSIZE);
close(fd);

inode bitmap  11100000
inodes        [d a:0 r:2][f a:1 r:1][f a:-1 r:1][][][][][]
data bitmap   11000000
data          [(.,0) (..,0) (k,1) (g,2)][g][][][][][][]

link("/k", "/b");

inode bitmap  11100000
inodes        [d a:0 r:2][f a:1 r:2][f a:-1 r:1][][][][][]
data bitmap   11000000
data          [(.,0) (..,0) (k,1) (g,2) (b,1)][g][][][][][][]

link("/b", "/t");

inode bitmap  11100000
inodes        [d a:0 r:2][f a:1 r:3][f a:-1 r:1][][][][][]
data bitmap   11000000
data          [(.,0) (..,0) (k,1) (g,2) (b,1) (t,1)][g][][][][][][]
```

```
unlink("/k");

inode bitmap  11100000
inodes        [d a:0 r:2][f a:1 r:2][f a:-1 r:1][][][][][]
data bitmap   11000000
data          [(.,0) (..,0) (g,2) (b,1) (t,1)][g][][][][][]
```

解释如下：

- 操作1只修改了inode位图，所以是creat()。查看1号inode,发现新建了一个文件，在0号数据块中查看新增加的条目，指示新建的文件名为“k”，所以操作1是creat(“/k”)
- 操作2只修改了inode位图，所以是creat()。查看2号inode,发现新建了一个文件，在0号数据块中查看新增加的条目，指示新建的文件名为“g”，所以操作2是creat(“/g”)
- 操作3修改了data位图，修改了1号inode（文件k）中的地址字段，增加了1号数据块，所以操作3是fd=open(“/k”,O_WRONLY|O_APPEND); write(fd, buf, BLOCKSIZE); close(fd)
- 操作4增加了1号inode（文件k）中的引用计数字段，所以是link()。在0号数据块中查看新增加的条目，指示新建的链接名为“b”，所以操作4是link(“/k”,“/b”)
- 操作5增加了1号inode（文件k）中的引用计数字段，所以是link()。在0号数据块中查看新增加的条目，指示新建的链接名为“t”，所以操作5是link(“/k”,“/t”)
- 操作6减小了1号inode（文件k）中的引用计数字段，所以是unlink()。在0号数据块中查看删除的条目，指示删除的链接名为“k”，所以操作6是unlink(“/k”)

## seed 20

执行如下指令

```
python3 ./vsfs.py -n 6 -s 20 -c
```

结果如下：

```
Initial state

inode bitmap  10000000
inodes        [d a:0 r:2][][][][][][][]
data bitmap   10000000
data          [(.,0) (..,0)][][][][][][][]
```

```
creat("/x");

inode bitmap  11000000
inodes        [d a:0 r:2][f a:-1 r:1][][][][][][]
data bitmap   10000000
data          [(.,0) (..,0) (x,1)][][][][][][][]

fd=open("/x", O_WRONLY|O_APPEND); write(fd, buf, BLOCKSIZE);
close(fd);

inode bitmap  11000000
inodes        [d a:0 r:2][f a:1 r:1][][][][][][]
data bitmap   11000000
data          [(.,0) (..,0) (x,1)][x][][][][][][]

creat("/k");

inode bitmap  11100000
inodes        [d a:0 r:2][f a:1 r:1][f a:-1 r:1][][][][][]
data bitmap   11000000
data          [(.,0) (..,0) (x,1) (k,2)][x][][][][][][]

creat("/y");

inode bitmap  11110000
inodes        [d a:0 r:2][f a:1 r:1][f a:-1 r:1][f a:-1 r:1][][][][]
data bitmap   11000000
data          [(.,0) (..,0) (x,1) (k,2) (y,3)][x][][][][][][]

unlink("/x");

inode bitmap  10110000
inodes        [d a:0 r:2][][f a:-1 r:1][f a:-1 r:1][][][][]
data bitmap   10000000
data          [(.,0) (..,0) (k,2) (y,3)][][][][][][][]

unlink("/y");

inode bitmap  10100000
inodes        [d a:0 r:2][][f a:-1 r:1][][][][][]
data bitmap   10000000
```

```
data          [(.,0) (..,0) (k,2)][][][][][][][]
```

解释如下：

- 操作1只修改了inode位图，所以是creat()。查看1号inode,发现新建了一个文件，在0号数据块中查看新增加的条目，指示新建的文件名为“x”，所以操作1是creat(“/x”)
- 操作2修改了data位图，修改了1号inode（文件x）中的地址字段，增加了1号数据块，所以操作2是fd=open(“/x”,O_WRONLY|O_APPEND); write(fd, buf, BLOCKSIZE); close(fd)
- 操作3只修改了inode位图，所以是creat()。查看2号inode,发现新建了一个文件，在0号数据块中查看新增加的条目，指示新建的文件名为“k”，所以操作3是creat(“/k”)
- 操作4只修改了inode位图，所以是creat()。查看3号inode,发现新建了一个文件，在0号数据块中查看新增加的条目，指示新建的文件名为“y”，所以操作4是creat(“/y”)
- 操作5修改了inode位图和data位图，删除了inode、数据块和目录块中的条目，所以是unlink()。发现删除的是1号inode（文件x），所以操作5是unlink(“/x”)
- 操作6修改了inode位图，删除了inode和目录块中的条目，所以是unlink()。发现删除的是3号inode（文件y），所以操作6是unlink(“/y”)

## 40.2

> 现在使用不同的随机种子（比如 *21*、*22*、*23*、*24*），但使用 *-r* 标志运行，这样做可以让你在显示操作时猜测状态的变化。关于 *inode* 和数据块分配算法，根据它们喜欢分配的块，你可以得出什么结论？

这道题建立在上一道题理解步骤含义的基础上。这次是让我们模拟计算机运行，给出不同时刻的状态。

### seed21

执行如下指令获取问题

```
python3 ./vsfs.py -n 6 -s 21 -r
```

分析：

- 操作1是`mkdir("/o")`，修改inode位图，增加一个inode用来存放新目录元数据，向存放新目录的目录块中增加一个条目，修改data位图，增加一个数据块用于存放

新目录的内容，更新相应inode中的引用计数

- 操作2是`creat("/b")`，修改inode位图，增加一个inode用来存放新文件元数据，向存放新文件的目录块中增加一个条目，更新相应inode中的引用计数
- 操作3是`creat("/o/q")`，修改inode位图，增加一个inode用来存放新文件元数据，向存放新文件的目录块中增加一个条目，更新相应inode中的引用计数
- 操作4是`fd=open("/b", O_WRONLY|O_APPEND); write(fd, buf, BLOCKSIZE); close(fd)`，修改data位图，增加一个数据块用于存放文件的新内容，修改inode中的数据块地址字段
- 操作5是`fd=open("/o/q", O_WRONLY|O_APPEND); write(fd, buf, BLOCKSIZE); close(fd)`，修改data位图，增加一个数据块用于存放文件的新内容，修改inode中的数据块地址字段
- 操作6是`creat("/o/j")`，修改inode位图，增加一个inode用来存放新文件元数据，向存放新文件的目录块中增加一个条目，更新相应inode中的引用计数

最终结果如下

```
Initial state

inode bitmap  10000000
inodes        [d a:0 r:2][][][][][][][]
data bitmap   10000000
data          [(.,0) (..,0)][][][][][][][]

mkdir("/o");

inode bitmap  11000000
inodes        [d a:0 r:3][d a:1 r:2][][][][][][]
data bitmap   11000000
data          [(.,0) (..,0) (o,1)][(.,1) (..,0)][][][][][][]

creat("/b");

inode bitmap  11100000
inodes        [d a:0 r:3][d a:1 r:2][f a:-1 r:1][][][][][]
data bitmap   11000000
data          [(.,0) (..,0) (o,1) (b,2)][(.,1) (..,0)][][][][][][]

creat("/o/q");

inode bitmap  11110000
inodes        [d a:0 r:3][d a:1 r:2][f a:-1 r:1][f a:-1 r:1][][][][]
```

```
data bitmap   11000000
data          [(.,0) (..,0) (o,1) (b,2)][(.,1) (..,0) (q,3)][][][][]
[][]

fd=open("/b", O_WRONLY|O_APPEND); write(fd, buf, BLOCKSIZE);
close(fd);

inode bitmap  11110000
inodes        [d a:0 r:3][d a:1 r:2][f a:2 r:1][f a:-1 r:1][][][][]
data bitmap   11100000
data          [(.,0) (..,0) (o,1) (b,2)][(.,1) (..,0) (q,3)][m][][]
[][][]

fd=open("/o/q", O_WRONLY|O_APPEND); write(fd, buf, BLOCKSIZE);
close(fd);

inode bitmap  11110000
inodes        [d a:0 r:3][d a:1 r:2][f a:2 r:1][f a:3 r:1][][][][]
data bitmap   11110000
data          [(.,0) (..,0) (o,1) (b,2)][(.,1) (..,0) (q,3)][m][j][]
[][][]

creat("/o/j");

inode bitmap  11111000
inodes        [d a:0 r:3][d a:1 r:2][f a:2 r:1][f a:3 r:1][f a:-1
r:1][][][]
data bitmap   11110000
data          [(.,0) (..,0) (o,1) (b,2)][(.,1) (..,0) (q,3) (j,4)]
[m][j][][][][]
```

### seed22

执行如下指令获取问题

```
python3 ./vsfs.py -n 6 -s 22 -r
```

最终结果如下

```
Initial state
```

```
inode bitmap  10000000
inodes        [d a:0 r:2][][][][][][][]
data bitmap   10000000
data          [(.,0) (..,0)][][][][][][][]

creat("/z");

inode bitmap  11000000
inodes        [d a:0 r:2][f a:-1 r:1][][][][][][]
data bitmap   10000000
data          [(.,0) (..,0) (z,1)][][][][][][][]

fd=open("/z", O_WRONLY|O_APPEND); write(fd, buf, BLOCKSIZE);
close(fd);

inode bitmap  11000000
inodes        [d a:0 r:2][f a:1 r:1][][][][][][]
data bitmap   11000000
data          [(.,0) (..,0) (z,1)][q][][][][][][]

unlink("/z");

inode bitmap  10000000
inodes        [d a:0 r:2][][][][][][][]
data bitmap   10000000
data          [(.,0) (..,0)][][][][][][][]

creat("/y");

inode bitmap  11000000
inodes        [d a:0 r:2][f a:-1 r:1][][][][][][]
data bitmap   10000000
data          [(.,0) (..,0) (y,1)][][][][][][][]

link("/y", "/s");

inode bitmap  11000000
inodes        [d a:0 r:2][f a:-1 r:2][][][][][][]
data bitmap   10000000
data          [(.,0) (..,0) (y,1) (s,1)][][][][][][][]
```

```
creat("/e");

inode bitmap  11100000
inodes        [d a:0 r:2][f a:-1 r:2][f a:-1 r:1][][][][][]
data bitmap   10000000
data          [(.,0) (..,0) (y,1) (s,1) (e,2)][][][][][][][]
```

### seed23

执行如下指令获取问题

```
python3 ./vsfs.py -n 6 -s 23 -r
```

最终结果如下

```
Initial state

inode bitmap  10000000
inodes        [d a:0 r:2][][][][][][][]
data bitmap   10000000
data          [(.,0) (..,0)][][][][][][][]

mkdir("/c");

inode bitmap  11000000
inodes        [d a:0 r:3][d a:1 r:2][][][][][][]
data bitmap   11000000
data          [(.,0) (..,0) (c,1)][(.,1) (..,0)][][][][][][]

creat("/c/t");

inode bitmap  11100000
inodes        [d a:0 r:3][d a:1 r:2][f a:-1 r:1][][][][][]
data bitmap   11000000
data          [(.,0) (..,0) (c,1)][(.,1) (..,0) (t,2)][][][][][][]

unlink("/c/t");

inode bitmap  11000000
inodes        [d a:0 r:3][d a:1 r:2][][][][][][]
```

```
data bitmap   11000000
data          [(.,0) (..,0) (c,1)][(.,1) (..,0)][][][][][][]

creat("/c/q");

inode bitmap  11100000
inodes        [d a:0 r:3][d a:1 r:2][f a:-1 r:1][][][][][]
data bitmap   11000000
data          [(.,0) (..,0) (c,1)][(.,1) (..,0) (q,2)][][][][][][]

creat("/c/j");

inode bitmap  11110000
inodes        [d a:0 r:3][d a:1 r:2][f a:-1 r:1][f a:-1 r:1][][][][]
data bitmap   11000000
data          [(.,0) (..,0) (c,1)][(.,1) (..,0) (q,2) (j,3)][][][][]
[][]

link("/c/q", "/c/h");

inode bitmap  11110000
inodes        [d a:0 r:3][d a:1 r:2][f a:-1 r:2][f a:-1 r:1][][][][]
data bitmap   11000000
data          [(.,0) (..,0) (c,1)][(.,1) (..,0) (q,2) (j,3) (h,2)][]
[][][][][]
```

## seed24

执行如下指令获取问题

```
python3 ./vsfs.py -n 6 -s 24 -r
```

最终结果如下

```
Initial state

inode bitmap  10000000
inodes        [d a:0 r:2][][][][][][][]
data bitmap   10000000
data          [(.,0) (..,0)][][][][][][][]
```

```
mkdir("/c");

inode bitmap  11000000
inodes        [d a:0 r:3][d a:1 r:2][][][][][][]
data bitmap   11000000
data          [(.,0) (..,0) (c,1)][(.,1) (..,0)][][][][][][]

creat("/c/t");

inode bitmap  11100000
inodes        [d a:0 r:3][d a:1 r:2][f a:-1 r:1][][][][][]
data bitmap   11000000
data          [(.,0) (..,0) (c,1)][(.,1) (..,0) (t,2)][][][][][][]

unlink("/c/t");

inode bitmap  11000000
inodes        [d a:0 r:3][d a:1 r:2][][][][][][]
data bitmap   11000000
data          [(.,0) (..,0) (c,1)][(.,1) (..,0)][][][][][][]

creat("/c/q");

inode bitmap  11100000
inodes        [d a:0 r:3][d a:1 r:2][f a:-1 r:1][][][][][]
data bitmap   11000000
data          [(.,0) (..,0) (c,1)][(.,1) (..,0) (q,2)][][][][][][]

creat("/c/j");

inode bitmap  11110000
inodes        [d a:0 r:3][d a:1 r:2][f a:-1 r:1][f a:-1 r:1][][][][]
data bitmap   11000000
data          [(.,0) (..,0) (c,1)][(.,1) (..,0) (q,2) (j,3)][][][][]
[][]

link("/c/q", "/c/h");

inode bitmap  11110000
inodes        [d a:0 r:3][d a:1 r:2][f a:-1 r:2][f a:-1 r:1][][][][]
data bitmap   11000000
```

```
data          [(.,0) (..,0) (c,1)][(.,1) (..,0) (q,2) (j,3) (h,2)][]
[][][][][]

wolf@wolf-VB:~/桌面/wolf/OS-homework/file-implementation$ python3
./vsfs.py -n 6 -s 24 -r -c
ARG seed 24
ARG numInodes 8
ARG numData 8
ARG numRequests 6
ARG reverse True
ARG printFinal False

Initial state

inode bitmap  10000000
inodes        [d a:0 r:2][][][][][][][]
data bitmap   10000000
data          [(.,0) (..,0)][][][][][][][]

mkdir("/z");

inode bitmap  11000000
inodes        [d a:0 r:3][d a:1 r:2][][][][][][]
data bitmap   11000000
data          [(.,0) (..,0) (z,1)][(.,1) (..,0)][][][][][][]

creat("/z/t");

inode bitmap  11100000
inodes        [d a:0 r:3][d a:1 r:2][f a:-1 r:1][][][][][]
data bitmap   11000000
data          [(.,0) (..,0) (z,1)][(.,1) (..,0) (t,2)][][][][][][]

creat("/z/z");

inode bitmap  11110000
inodes        [d a:0 r:3][d a:1 r:2][f a:-1 r:1][f a:-1 r:1][][][][]
data bitmap   11000000
data          [(.,0) (..,0) (z,1)][(.,1) (..,0) (t,2) (z,3)][][][][]
[][]
```

```
fd=open("/z/z", O_WRONLY|O_APPEND); write(fd, buf, BLOCKSIZE);
close(fd);

inode bitmap  11110000
inodes        [d a:0 r:3][d a:1 r:2][f a:-1 r:1][f a:2 r:1][][][][]
data bitmap   11100000
data          [(.,0) (..,0) (z,1)][(.,1) (..,0) (t,2) (z,3)][y][][]
[][][]

creat("/y");

inode bitmap  11111000
inodes        [d a:0 r:3][d a:1 r:2][f a:-1 r:1][f a:2 r:1][f a:-1
r:1][][][]
data bitmap   11100000
data          [(.,0) (..,0) (z,1) (y,4)][(.,1) (..,0) (t,2) (z,3)]
[y][][][][][]

fd=open("/y", O_WRONLY|O_APPEND); write(fd, buf, BLOCKSIZE);
close(fd);

inode bitmap  11111000
inodes        [d a:0 r:3][d a:1 r:2][f a:-1 r:1][f a:2 r:1][f a:3
r:1][][][]
data bitmap   11110000
data          [(.,0) (..,0) (z,1) (y,4)][(.,1) (..,0) (t,2) (z,3)]
[y][v][][][][]
```

### 关于 **inode** 和数据块分配算法

分析上述情境，我们不难发现：分配算法会使用最近可分配的 inode 与数据块。

针对其余随机数种子的做法都是相同的，为节约篇幅，在此使用-c标志查看答案。通过-c查看答案验证，发现与分析一致。

## 40.3

> 现在将文件系统中的数据块数量减少到非常少（比如两个），并用 *100* 个左右的请求来运行模拟器。在这种高度约束的布局中，哪些类型的文件最终会出现在文件系统中？什么类型的操作会失败？

执行如下指令：

```
python3 ./vsfs.py -d 2 -c -n 100
```

结果如下：

```
Initial state

inode bitmap  10000000
inodes        [d a:0 r:2][][][][][][][]
data bitmap   10
data          [(.,0) (..,0)][]

mkdir("/g");
File system out of data blocks; rerun with more via command-line
flag?
```

数据块太少了，连第一个文件都无法创建。我们可以试着把数据块给多一点，比如给5个。

```
Initial state

inode bitmap  10000000
inodes        [d a:0 r:2][][][][][][][]
data bitmap   10000
data          [(.,0) (..,0)][][][][]

mkdir("/g");

inode bitmap  11000000
inodes        [d a:0 r:3][d a:1 r:2][][][][][][]
data bitmap   11000
data          [(.,0) (..,0) (g,1)][(.,1) (..,0)][][][]

creat("/q");

inode bitmap  11100000
inodes        [d a:0 r:3][d a:1 r:2][f a:-1 r:1][][][][][]
data bitmap   11000
data          [(.,0) (..,0) (g,1) (q,2)][(.,1) (..,0)][][][]

creat("/u");
```

```
inode bitmap  11110000
inodes        [d a:0 r:3][d a:1 r:2][f a:-1 r:1][f a:-1 r:1][][][][]
data bitmap   11000
data          [(.,0) (..,0) (g,1) (q,2) (u,3)][(.,1) (..,0)][][][]

link("/u", "/x");

inode bitmap  11110000
inodes        [d a:0 r:3][d a:1 r:2][f a:-1 r:1][f a:-1 r:2][][][][]
data bitmap   11000
data          [(.,0) (..,0) (g,1) (q,2) (u,3) (x,3)][(.,1) (..,0)][]
[][]

mkdir("/t");

inode bitmap  11111000
inodes        [d a:0 r:4][d a:1 r:2][f a:-1 r:1][f a:-1 r:2][d a:2
r:2][][][]
data bitmap   11100
data          [(.,0) (..,0) (g,1) (q,2) (u,3) (x,3) (t,4)][(.,1)
(..,0)][(.,4) (..,0)][][]

creat("/g/c");

inode bitmap  11111100
inodes        [d a:0 r:4][d a:1 r:2][f a:-1 r:1][f a:-1 r:2][d a:2
r:2][f a:-1 r:1][][]
data bitmap   11100
data          [(.,0) (..,0) (g,1) (q,2) (u,3) (x,3) (t,4)][(.,1)
(..,0) (c,5)][(.,4) (..,0)][][]

unlink("/x");

inode bitmap  11111100
inodes        [d a:0 r:4][d a:1 r:2][f a:-1 r:1][f a:-1 r:1][d a:2
r:2][f a:-1 r:1][][]
data bitmap   11100
data          [(.,0) (..,0) (g,1) (q,2) (u,3) (t,4)][(.,1) (..,0)
(c,5)][(.,4) (..,0)][][]

mkdir("/g/w");
```

```
inode bitmap  11111110
inodes        [d a:0 r:4][d a:1 r:3][f a:-1 r:1][f a:-1 r:1][d a:2
r:2][f a:-1 r:1][d a:3 r:2][]
data bitmap   11110
data          [(.,0) (..,0) (g,1) (q,2) (u,3) (t,4)][(.,1) (..,0)
(c,5) (w,6)][(.,4) (..,0)][(.,6) (..,1)][]

fd=open("/g/c", O_WRONLY|O_APPEND); write(fd, buf, BLOCKSIZE);
close(fd);
File system out of data blocks; rerun with more via command-line
flag?
```

因为mkdir()和open(), write(), close()需要数据块，而creat()、link()、unlink()不需要数据块，所以mkdir()和open(), write(), close()操作会失败，creat()、link()、unlink()操作不会失败。

## 40.4

> 现在做同样的事情，但针对 *inodes*。只有非常少的 *inode*，什么类型的操作才能成功？哪些通常会失败？文件系统的最终状态可能是什么？

执行如下指令

```
python3 ./vsfs.py -i 5 -c -n 100
```

结果如下：

```
Initial state

inode bitmap  10000
inodes        [d a:0 r:2][][][][]
data bitmap   10000000
data          [(.,0) (..,0)][][][][][][][]

mkdir("/g");

inode bitmap  11000
inodes        [d a:0 r:3][d a:1 r:2][][][]
data bitmap   11000000
```

```
data          [(.,0) (..,0) (g,1)][(.,1) (..,0)][][][][][][]

creat("/q");

inode bitmap  11100
inodes        [d a:0 r:3][d a:1 r:2][f a:-1 r:1][][]
data bitmap   11000000
data          [(.,0) (..,0) (g,1) (q,2)][(.,1) (..,0)][][][][][][]

creat("/u");

inode bitmap  11110
inodes        [d a:0 r:3][d a:1 r:2][f a:-1 r:1][f a:-1 r:1][]
data bitmap   11000000
data          [(.,0) (..,0) (g,1) (q,2) (u,3)][(.,1) (..,0)][][][][]
[][]

link("/u", "/x");

inode bitmap  11110
inodes        [d a:0 r:3][d a:1 r:2][f a:-1 r:1][f a:-1 r:2][]
data bitmap   11000000
data          [(.,0) (..,0) (g,1) (q,2) (u,3) (x,3)][(.,1) (..,0)][]
[][][][][][]

mkdir("/t");
File system out of inodes; rerun with more via command-line flag?
```

因为mkdir()和creat()需要inode，而open(), write(), close() 、link()、unlink()不需要inode，所以mkdir()和creat()操作会失败，open(), write(), close() 、link()、unlink()操作不会失败。